

Generative KI für 3D-Medizinbildgebung mittels Diffusion Models

Generative AI for 3D Medical Imaging using Diffusion Models

Studienarbeit

vorgelegt von

Sven Beller

2026

Matrikelnummer, Kurs

5102447, TINF23

Betreuer

Malte Tölle

Sperrvermerk

TODO: Anpassen

Die vorliegende Arbeit beinhaltet interne vertrauliche Informationen des Unternehmens ZIEHL-ABEGG SE. Sie ist nur für die Beteiligten am Begutachtungs- und Evaluationsprozess bestimmt. Die Weitergabe des Inhalts der Arbeit im Gesamten oder in Teilen sowie das Anfertigen von Kopien oder Abschriften - auch in digitaler Form - sind grundsätzlich untersagt. Ausnahmen bedürfen der schriftlichen Genehmigung des Unternehmens ZIEHL-ABEGG SE.

Dieser Sperrvermerk gilt unbefristet.

Künzelsau, 1. Januar 2025

Sven Beller

Erklärung

Ich, **Name**, versichere hiermit, dass ich meine **Hier T3_2000/T3_3100 etc** mit dem Thema **Hier Thema** selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich erkläre zudem, dass ich generative Systeme (Text-, Bild-, Codeerstellung etc.) bei der Erstellung der vorliegenden wissenschaftlichen Arbeit in der nachfolgend beschriebenen Art und dem nachfolgend beschriebenen Umfang verwendet habe.

Ort, Datum

Name

Genutzter Funktionsumfang

Themenerfassung und Strukturierung (Aufgabenstellung/Präzisierung, Forschungsansatz, Gliederung)

Produkt/Tool	Beschreibung / Verwendungsweise
Tool	Ideenfindung und Grobgliederung der Arbeit. Beispiel: Welche Themen existieren über Thema?

Quellenrecherche, -auswahl und -auswertung (durch AI gefundene Quellen; Vorgehen der weiteren Recherche)

Produkt/Tool	Beschreibung / Verwendungsweise
Tool	Vorschläge relevanter Publikationen, anschließende manuelle Validierung. Beispiel: Welche Bücher behandeln das Thema Thema?

Formale Gestaltung, insbesondere Sprache (Prompts/Eingaben; Textgenerierung, Korrektur, Paraphrasieren, Übersetzen, kreatives Schreiben)

Produkt/Tool	Beschreibung / Verwendungsweise
Tool	Grammatik-/Stilkorrektur einzelner Absätze; keine inhaltliche Neuerstellung. Beispiel: Ist der folgende Satz grammatikalisch richtig?

Abstract

This is the abstract in English.

Zusammenfassung

Dies ist die Zusammenfassung auf Deutsch.

Inhaltsverzeichnis

Abbildungsverzeichnis	IX
Tabellenverzeichnis	X
Nomenklatur	XII
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	1
1.3 Vorgehensweise	1
2 Medizinische Bildgebung	2
2.1 Bildgebende Verfahren	2
2.2 NifTI	2
3 Künstliche Intelligenz	3
3.1 Defintion Künstliche Intelligenz	3
3.1.1 Defintion Intelligenz	4
3.1.2 Schwache und Starke KI	4
3.2 Maschinelles Lernen	5
3.3 Artificial Neural Networks	5
3.3.1 Perceptron	6
3.4 Unterscheidung zwischen Diskriminative und Generative Modelle	7
3.4.1 Diskriminative Modelle	7
3.4.2 Generative Modelle	7
4 Stand der Forschung	9
4.1 Generative Modelle in der medizinischen Bildgebung	9
4.2 Vergleich bestehender Ansätze	9

5	Deep Generative Model	10
5.1	GANs	10
5.2	VAE	10
5.3	Diffusion Model	10
5.3.1	Markov-Ketten	11
5.3.2	Noise-Schedule	11
5.3.3	Vorwärtsprozess	12
5.3.4	Rückwärtsprozess	13
5.3.5	Sampling	14
5.4	Grundlegende Architekturen	16
5.4.1	Convolutional Neural Network	16
5.4.2	U-Net	18
5.5	Einsatzmöglichkeiten in der Medizin	20
6	Datenqualität und ethische Aspekte	21
6.1	Definition synthetischer Daten	21
6.2	Ethische Aspekte synthetischer medizinischer Daten	22
6.3	Risiken und Grenzen synthetischer Daten	22
6.4	Qualitätskriterien für synthetische medizinische Bilddaten	23
6.4.1	Visuelle Bewertungskriterien	23
6.4.2	Mathematische Bewertungsmetriken	23
6.4.3	Peak Signal-to-Noise Ratio	25
6.4.4	Learned Perceptual Image Patch Similarity	26
6.4.5	Dice	26
7	Implementierung und experimenteller Aufbau	27
7.1	Entwicklungsumgebung und Hardware	27
7.2	Datenvorverarbeitung	28
7.3	Modellarchitektur	28
7.4	Baseline-Konfiguration	29
7.5	Trainingsverfahren und Hyperparameter	29
7.6	Ablationsstudien	30
7.7	Aufgezeichnete Werte	31
7.8	Projektmethodik	31

8 Ergebnis	32
9 Schlussfolgerung	33
9.1 Fazit	33
9.2 Ausblick	33
Literaturverzeichnis	XV
A Anhang A	XXI

Abbildungsverzeichnis

2.1	NifTI-Header [3, S. 76]	2
5.1	Markov-Kette erster Ordnung [17, S. 609]	11
5.2	Darstellung der Vorwärts- und Rückwärtsprozesse	12
5.3	Convolutional [27, S. 6]	17
5.4	CNN	18
5.5	U-Net Architektur [32, S. 2]	18
6.1	Darstellung der Fréchet Inception Distance (FID)-Berechnung [50, S. 130] . .	24
6.2	Darstellung der Structural Similarity Index Measure (SSIM)-Berechnung [53, S. 5]	25

Tabellenverzeichnis

Nomenklatur

Lateinische Buchstaben

A	m^2	Fläche
$\vec{a}$	m/s^2	Beschleunigung
a	m/s	Schallgeschwindigkeit
c_p	J/(kgK)	spezifische isobare Wärmekapazität
c_v	J/(kgK)	spezifische isochore Wärmekapazität
E	J	innere Energie
e	J/kg	spezifische innere Energie
g	m/s^2	Erdbeschleunigung
H	J	Enthalpie
h	J/kg	spezifische Enthalpie
k	J/K	Boltzmann-Konstante $(k = 1,3807 \cdot 10^{-23})$
m	kg	Masse
N_A	mol^{-1}	Avogadro-Konstante $(N_A = 6,0021318 \cdot 10^{23})$
$\vec{n}$	-	nach außen gerichteter Normaleneinheitsvektor
n	-	Polytropenexponent
P	J/s	Leistung
p	N/m^2	Druck
Q	J	Wärme
$\dot{Q}$	J/s	Wärmestrom
q	J/kg	bezogene Wärme
$\dot{q}$	J/(kg s)	bezogener Wärmestrom
R	J/(kgK)	spezielle Gaskonstante
R_m	J/(molK)	universelle Gaskonstante $(R_m = 8,31441)$
T	K	Temperatur
t	s	Zeit
u, v, w	m/s	Geschwindigkeitskomponenten im kartesischen Koordinatensystem
$\vec{V}$	m/s	Geschwindigkeitsvektor
V	m^3	Volumen
v	m^3/kg	spezifisches Volumen
W	J	Arbeit
x, y, z	m	kartesische Koordinaten
z	m	Höhe

Griechische Buchstaben

η	Pa s	dynamische Viskosität (Scherviskosität)
κ	-	Isentropenexponent
ν	m^2/s	kinematische Viskosität
ν	-	Polytropenverhältnis
ρ	kg/m^3	Dichte
τ_w	N/m^2	Wandschubspannung
$\vec{\omega}$	s^{-1}	Winkelgeschwindigkeit

Tiefgestellte Indizes

0	Ruhegröße
n	normal
t	tangential

Hochgestellte Indizes

*	kritischer Zustand (LAVAL-Zustand)
---	------------------------------------

Dimensionslose Kennzahlen

Ma	$:= \ \vec{V}\ /a$	Mach-Zahl
Ma*	$:= \ \vec{V}\ /a^*$	kritische Mach-Zahl
Re	$:= \ \vec{V}\ L/\nu$	Reynolds-Zahl

Operatoren, Funktionen und Symbole

Abkürzungen

KI	Künstliche Intelligenz
NLP	Natural Language Processing
DGM	Deep generative model
NIfTI	Neuroimaging Informatics Technology Initiative
NIH	National Institutes of Health
CNN	Convolutional Neural Network
FID	Fréchet Inception Distance
IS	Inception score
GANs	Generative Adversarial Networks

IQMs	Image Quality Metrics
FR	full-reference
RR	reduced-reference
NR	no-reference
SSIM	Structural Similarity Index Measure
PSNR	Peak Signal-to-Noise Ratio
MSE	Mean squared Error
ReLU	Rectified Linear Unit
ANN	Artificial Neural Networks
GAN	Generative Adversary Network
VAE	Variational Autoencoder Network
DDPM	Diffusion Denoising Probabilistic Model
DDIM	Diffusion Denoising Implicit Model
LPIPS	Learned Perceptual Image Patch Similarity
ODE	Ordinary Differential Equation
UniPC	Unified Predictor-Corrector

1 Einleitung

TODO: neu schreiben/bisschen abändern

In der Medizin werden mithilfe von 3D-Bildgebung bessere Diagnosen von Krankheiten und genauere Eingriffe durchgeführt. Dabei bringt die steigende Entwicklung von generativer KI neue Möglichkeiten für medizinische Bilddaten.

Diffusion Models haben sich als leistungsfähige Methode zur realistischen Bildsynthese etabliert. In der datengetriebenen Medizin besteht ein wachsender Bedarf an hochwertigen, vielfältigen und datenschutzkonformen Bilddaten, insbesondere für das Training von KI-Systemen. Dieses Projekt widmet sich der Entwicklung eines Diffusion Models zur Generierung synthetischer 3D-medizinischer Volumendaten, die für Forschungszwecke, Augmentierung und Trainingsprozesse genutzt werden können.

1.1 Problemstellung

Für generative KI-Modelle besteht ein wachsender Bedarf an Daten, um ein Model darauf zu trainieren. Allerdings ist die Verfügbarkeit dieser Daten eingeschränkt durch Datenschutzbestimmungen und an der Bildakquise von seltenen Krankheiten. Durch diese Limitierung besteht eine Erschwerung des Trainieren solcher Modelle. Es besteht daher ein Bedarf an synthetischen aber realistischen Bilddaten, die sowohl qualitativ und ethisch vertretbar sind.

1.2 Zielsetzung

Ziel dieser Studienarbeit ist die Entwicklung eines generativen KI-Systems, das mittels Diffusion Models realistische 3D-medizinische Bilddaten erzeugt.

1.3 Vorgehensweise

2 Medizinische Bildgebung

2.1 Bildgebende Verfahren

2.2 NIfTI

Das Neuroimaging Informatics Technology Initiative (NIfTI)-Format (.nii, .nii.gz) wurde in den frühen 2000er Jahren vom National Institutes of Health (NIH) entwickelt und hat sich als Standardformat in der neuroimaging-basierten Forschung etabliert. Es ist speziell für die Verarbeitung volumetrischer 3D-Bilddaten ausgelegt und daher geeignet für beispielsweise MRT-Gehirnuntersuchungen [1, S. 203]. NIfTI ist ein Rasterformat, das Daten in dreidimensionaler Form als Voxel speichert also als Pixel mit einer definierten Breite, Höhe und Tiefe [2, S. 4]. Dabei werden in den ersten drei Dimensionen die räumlichen Koordinaten x , y und z gespeichert, während die vierte Dimension für den Zeitpunkt t reserviert ist [3, S. 76]. Der Header einer NIfTI-Datei umfasst im .nii-Format 352 Bytes [3, S. 76]. In Abb. 2.1 ist ein Beispiel-Header zu sehen, welcher zeigt, welche Metadaten gespeichert werden. So ist beispielsweise im Parameter ImageSize die Auflösung des MRT-Scans abzulesen, in diesem Fall 256x156x256. Im Gegensatz zu Formaten

Parameters	Value
Filename	group00001_T1w_CSF_probability.nii
Filesize	40894816
Description	'DAD210715'
ImageSize	[256 156 256]
xPixelDimensions	[1 1.3000 1]
Datatype	'single'
BitsPerPixel	32
SpaceUnits	'Millimeter'
TimeUnits	'Second'
MultiplicativeScaling	1

Abb. 2.1: NIfTI-Header [3, S. 76]

wie JPEG, das eine verlustbehaftete Kompression verwendet und dadurch relevante Bildinformationen beeinträchtigen kann, unterstützt NIfTI eine verlustfreie Kompression [1, S. 203].

3 Künstliche Intelligenz

TODO: Titel gegebenenfalls anpassen **Schreibe: Schreiben worüber in diesem Kapitel behandelt wird**

3.1 Defintion Künstliche Intelligenz

Der Begriff Künstliche Intelligenz (KI) kann je nach Anwendungsgebiet eine andere Definition haben. Ein Buch beschreibt die KI als ein Forschungsgebiet [4, S. 5]. Jedoch wird die KI in dieser Studienarbeit beschrieben als eine Methode, dass Computern die Intelligenz des Menschen vorgibt und somit menschenähnlich zu denken [5, S. 3] [6, S. 42].

TODO: Agenten nochmal neu definieren. Siehe S. 17 Knowledge Science - Grundlagen

3.1.1 Defintion Intelligenz

Die Intelligenz kann als eine Sammlung von Eigenschaften verstanden werden und es ist das Ziel dass diese Eigenschaften nachgebaut werden. Wissenschaftler in der KI haben die folgenden Eigenschaften zu der Intelligenz definiert: [7, S. 1 f.]

- **Wahrnehmung:** Die Beobachtung seiner Umwelt mithilfe von Sensoren.
- **Schlussfolgern:** Aus bekannten Informationen rationale Entscheidungen ziehen auch bei Unsicherheit.
- **Handeln:** Aus bekannten Informationen gezielt handeln und Werkzeuge nutzen oder entwickeln.
- **Planung und Problemlösung:** Schritte planen, um ein Ziel zu erreichen oder ein Problem zu lösen.
- **Kommunikation:** Mit anderen Systemen oder Menschen kommunizieren.
- **Anpassungsfähigkeit und Lernen:** Aus Erfahrungen lernen und sich an neue Situationen anpassen.
- **Autonomie:** Eigene Ziele setzen und selbst entscheiden, wie man sie erreicht.
- **Kreativität:** Neue Wege zur Lösung von Problemen finden.
- **Reflexion und Bewusstheit:** Über eigene Entscheidungen und Prozesse reflektieren.
- **Ästhetik:** Bei Entscheidungen auch ästhetische Aspekte berücksichtigen.
- **Organisation:** Mit anderen zusammenarbeiten und sich abstimmen.

Schreibe: Über die Punkte angehen, wie diese die Intelligenz beschreibe **TODO: Bearbeiten** Anschließend zu der Intelligenz zählt das rationale Denken. Beim rationalen Denken wird auf Basis des bekannten Informationen auch mit Unsicherheit, die bestmögliche Entscheidung getroffen. Das System soll auch auf diese Entscheidungen handeln und selbstständig weiterlernen. Ein solches System wird als *intelligenter Agent* bezeichnet [8, S. 4].

3.1.2 Schwache und Starke KI

Wie intelligent eine KI ist, wird unter Wissenschaftler zwischen einer *schwachen (weak)* / *beschränkten (narrow)* und *starken (strong)* / *allgemeine (general)* KI unterschieden [9, S. 1]. Eine schwache KI ist spezifisch für eine Domäne entwickelt, die die menschliche Intelligenz dort simuliert [9, S. 1] [10, S. 7]. Diese können ein oder wenige spezifische Probleme sehr gut lösen. Allerdings kann die schwache KI bekannte Lösungsmuster auf

neue Probleme nicht in andere Domänen übertragen [10, S. 7]. Momentan können alle KI-Systeme als eine schwache KI bewertet werden, da sie jeweils spezifisch zu ihrer Domäne sind. Eine beispiel Domäne ist die Natural Language Processing (NLP) [9, S. 1]. Die schwache KI wird noch mal in zwei weitere Kategorien verfasst und unterscheiden sich dabei, mit welcher Methode sie trainiert werden. Diese sind die *symbolische KI* und *nicht-symbolische KI*. In der symbolischen KI wird diese mithilfe von Deduktion trainiert. Heißt auf logische Regeln. Während die nicht-symbolische KI mit Induktion also anhand von vorhandene Daten lernt [5, S. 4 f.]. Eine starke KI hingegen besitzt die Fähigkeit die menschenliche Intelligenz zu emulieren. Dies bedeutet, dass sie selbständig fungiert und somit auch selbstbewusst ist. Dabei kann die starke KI Probleme lösen, ohne davor dafür trainiert zu sein [10, S. 7] [11, S. 17]. Dies würde dann zu einer technologischen Singularität führen und einer Superintelligenz [9, S. 1] [10, S. 7] [11, S. 17]. Eine technologische Singularität ist der Punkt, an dem die KI nicht mehr vom Menschen kontrolliert werden kann [11, S. 18]. Allerdings gibt bis jetzt keine starke KI und somit ein theoretisches Konzept. Es wird jedoch in Science-Fiction-Werke angewendet [9, S. 1] [11, S. 17 f.] [5, S. 4].
TODO: Neu durchlesen

3.2 Maschinelles Lernen

Zu diesen Methoden gehört das maschinelle Lernen, wobei der Computer selbständig Wissen generiert aus Erfahrungen. Dabei wird der Computer durch Beispiele trainiert worraus dieser dann Muster abgeleitet werden. Diese Muster werden dann auf die jeweilige Situation angewendet [6, S. 42].

3.3 Artificial Neural Networks

Ein Artificial Neural Networks (ANN) (engl. *Künstliches Neuronales Netzwerk*) ist ein rechnergestütztes Modell, das die Informationsverarbeitung des menschlichen Gehirns nachbildet. Inspiriert durch das biologische Nervensystem besteht es aus einer Vielzahl miteinander verbundener künstlicher Neuronen, die Informationen schichtweise empfangen, verarbeiten und weiterleiten [12, S. 52] [13, S. 82]. Durch das Anpassen ihrer internen Parameter ist ein ANN in der Lage, komplexe Muster in Daten zu erkennen und Vorhersagen zu treffen. Ein ANN erlaubt somit einer Maschine Aufgaben zu erledigen, die eine menschliche Intelligenz benötigt hätten. Darunter gehören beispielsweise die Bilderkennung und NLP [12, S. 52]. Weiterhin ist ein ANN robust gegenüber vertauschten Daten [13, S. 83]. **TODO: mehr ausschreiben** Ein einzelner künstlicher Neuron kann mathematisch wie folgt beschrieben:

TODO: Feedforward und Recurrent Neural Network

3.3.1 Perceptron

Das Perceptron wurde eingeführt von Rosenblatt im Jahr 1957 und gilt als die einfachste Form eines ANN. Es handelt sich um einen elektronischen Mechanismus, der dazu fähig ist, Muster zu erkennen. Seine Ausgaben hängen von Wahrscheinlichkeiten ab statt von einem deterministischen Prozess. Ein Modell, das optische bzw. visuelle Muster erkennt, wird als *Photoperceptron* bezeichnet [14, S. 2]. Mathematisch wird ein Perceptron wie folgt dargestellt [15, S. 39]:

$$f(x) = \text{sign}(w \cdot x + b) \quad (3.1)$$

Dabei bezeichnen $w \subseteq \mathbb{R}^n$ den Gewichtsvektor und $b \in \mathbb{R}$ den Bias-Term. Das Produkt $w \cdot x$ verknüpft den Gewichtsvektor mit dem Eingabevektor. Die Aktivierungsfunktion sign ist eine Vorzeichenfunktion, die wie folgt definiert ist [15, S. 40]:

$$\text{sign}(x) = \begin{cases} +1, & x \geq 0 \\ -1, & x < 0 \end{cases} \quad (3.2)$$

Das Perceptron ist ein lineares Klassifikationsmodell und gehört zu den diskriminativen Modellen [15, S. 40].

Somit kann ein einzelnes Perceptron als Boolesche Funktion verwendet werden und die logischen Operationen AND, OR, NAND und NOR darstellen. Dabei lassen sich AND und OR als sogenannter *m-of-n*-Funktionen auffassen, bei denen mindestens m von n Eingaben wahr sein müssen. Damit ein Perceptron als ein AND dargestellt werden können... [13, S. 87]. Allerdings kann die Boolesche Funktionen XOR nicht von einem einzelnen Perceptron dargestellt werden [13, S. 87]. **TODO: nochmal durchlesen** Diese Einschränkung wird durch ein **MLP! (MLP!)** überwunden. Jede Boolesche Funktion lässt sich durch ein Netzwerk aus Perceptronen mit lediglich zwei Schichten darstellen, wobei die Eingaben zunächst an mehrere Einheiten weitergeleitet und deren Ausgaben anschließend in einer zweiten Stufe zusammengeführt werden [13, S. 88].

Eine wesentliche Einschränkung des Perceptrons besteht darin, dass es ausschließlich *linear separierbare* Probleme lösen kann. **minsky_perceptrons_1969** zeigten formal, dass bereits einfache logische Funktionen wie das XOR-Problem von einem einzelnen Perceptron nicht lösbar sind [**minsky_perceptrons_1969**]. Diese Limitation wird durch sogenannte *Multi-Layer Perceptrons* (MLPs) überwunden, bei denen mehrere Neuronen in verborgenen Schichten angeordnet werden und mittels des Backpropagation-Algorithmus auch nichtlineare Zusammenhänge modelliert werden können.

Es besteht aus einem einzelnen künstlichen Neuron, das mehrere Eingabewerte $x_1, x_2, \dots, x_n$ entgegennimmt, diese jeweils mit einem zugehörigen Gewicht $w_1, w_2, \dots, w_n$ multipliziert und aufsummiert. Zusätzlich wird ein Bias-Term b addiert, der als Schwellenwertverschiebung dient. Auf die resultierende gewichtete Summe wird eine Aktivierungsfunktion f angewendet, sodass sich die Ausgabe y wie folgt ergibt:

$$y = f\left(\sum_{i=1}^n w_i \cdot x_i + b\right) \quad (3.3)$$

Beim klassischen Perceptron ist die Aktivierungsfunktion eine Heaviside-Stufenfunktion, die den Wert 1 ausgibt, wenn die gewichtete Summe einen Schwellenwert überschreitet, und andernfalls den Wert 0 [rosenblatt_perceptron_1958].

Das Perceptron lernt durch iterative Anpassung seiner Gewichte. Wird eine Eingabe falsch klassifiziert, so werden die Gewichte proportional zum Fehler korrigiert. Dieser Vorgang wird als Perceptron-Lernregel bezeichnet und lässt sich formal beschreiben als:

$$w_i^{(t+1)} = w_i^{(t)} + \eta \cdot (y_{\text{soll}} - y_{\text{ist}}) \cdot x_i \quad (3.4)$$

wobei η die Lernrate bezeichnet, die die Schrittweite der Gewichtsanzpassung steuert [13, S. 86 ff.].

Schreibe: Was ist Intelligenz

Schreibe: Je weniger Eigenschaften gebracuth werden desto besser können Ergebnisse sein

3.4 Unterscheidung zwischen Diskriminative und Generative Modelle

Modelle des maschinellen Lernens werden häufig in zwei Kategorien unterteilt: diskriminative und generative Modelle. Beide verfolgen unterschiedliche Ziele und unterscheiden sich darin, was sie aus den gegebenen Daten lernen.

3.4.1 Diskriminative Modelle

Diskriminative Modelle versuchen Eingabedaten einer bestimmten Klasse zuzuordnen. Sie modellieren dazu direkt die Wahrscheinlichkeit $p(y|x)$ des Labels y gegeben der Eingabe x oder erlernen eine direkte Abbildung von der Eingabe x auf das Label y [vgl. 16, S. 1]. Auf diese Weise lernen sie eine Entscheidungsgrenze zwischen den Klassen, ohne die gemeinsame Verteilung $p(x, y)$ zu modellieren [vgl. 17, S. 43].

Beispiele für diskriminative Modelle sind die logistische Regression [17, S. 205] sowie Feed-forward Network Functions [17, S. 227]. Sie sind dadurch allerdings nicht in der Lage, neue Daten zu generieren, da ihnen die Information über die gemeinsame Verteilung $p(x, y)$ fehlt. Ein diskriminatives Modell könnte beispielsweise lernen, ob ein Röntgenbild eine Pathologie zeigt oder nicht, ohne zu verstehen, wie ein Röntgenbild grundsätzlich aufgebaut ist.

3.4.2 Generative Modelle

Generative Modelle hingegen sind in der Lage neue synthetische Daten zu generieren. Sie lernen, wie die Daten strukturiert sind und können durch das Erlernen dieser Struktur neue Daten generieren, die dieser Struktur folgen [vgl. 18, S. 112]. Generative Modelle

lernen dazu die gemeinsame Wahrscheinlichkeitsverteilung $p(x, y)$ der Eingabe x und der zugehörigen Labels y [vgl. 16, S. 1]

Generative Model werden auch als Deep generative model (DGM)s bezeichnet. Unter ein DGM wird ein neuronales Netzwerk verstanden, dass die Wahrscheinlichkeitsverteilung von hoch dimensionalen Daten, wie Bilder, Text oder Audio, lernt [19, S. 16]. Sie besitzen eine hohe Anzahl von versteckten Schichten aus Neuronen, um das Erlernen der hochdimensionalen Datenverteilungen zu ermöglichen. Allerdings sind sie sehr rechenaufwendig und die Performanz ist stark abhängig von *Hyperparametern*. Hyperparameter beziehen sich auf Einstellungen, die vor dem Training festgelegt werden und den Lernprozess steuern. Dazu gehören die Netzwerkarchitektur, die Wahl der Zielfunktion, Regularisierungstechniken sowie Trainingsalgorithmen [20, S. 2]. Somit erlernen generative Modelle wie beispielsweise ein Röntgenbild aufgebaut ist und auf dieser Basis neue, synthetische Röntgenbilder erzeugen.

4 Stand der Forschung

TODO: Benötigt?

4.1 Generative Modelle in der medizinischen Bildgebung

4.2 Vergleich bestehender Ansätze

5 Deep Generative Model

5.1 GANs

Generative Adversary Network (GAN)s bestehen aus zwei neuronalen Netzwerken, die gegeneinander trainiert werden: einem *Generator* G und einem *Diskriminator* D . Der Generator erzeugt aus einem zufälligen Rauschen $z \sim p_z(z)$ synthetische Daten $G(z)$, während der Diskriminator lernt, echte Daten $x \sim p_{data}(x)$ von gefälschten zu unterscheiden. Dieses Wettbewerbsprinzip wird als *Minimax-Spiel* bezeichnet und durch folgende Zielfunktion beschrieben:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (5.1)$$

Der Diskriminator D versucht, $D(x) \approx 1$ für echte und $D(G(z)) \approx 0$ für gefälschte Daten zu erreichen. Der Generator G hingegen versucht, den Diskriminator zu täuschen, sodass $D(G(z)) \approx 1$. Im Idealfall konvergiert das Training zu einem Nash-Gleichgewicht, bei dem G die wahre Datenverteilung p_{data} approximiert.

5.2 VAE

Ein Variational Autoencoder Network (VAE) ist ein generatives Modell, das Daten in einen komprimierten *latenten Raum* z kodiert und aus diesem neue Daten rekonstruiert. Im Gegensatz zu klassischen Autoencodern kodiert der VAE nicht einen festen Punkt, sondern eine Wahrscheinlichkeitsverteilung $q_\phi(z | x) = \mathcal{N}(\mu, \sigma^2 I)$, aus der dann Samples gezogen werden. Das Training minimiert die folgende Verlustfunktion, den *Evidence Lower Bound* (ELBO):

$$\mathcal{L}_{VAE} = \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)]}_{\text{Rekonstruktionsfehler}} - \underbrace{D_{KL}(q_\phi(z | x) \parallel p(z))}_{\text{Regularisierung}} \quad (5.2)$$

Der erste Term minimiert den Rekonstruktionsfehler – das Modell soll die Eingabe x möglichst gut wiederherstellen. Der zweite Term, die *KL-Divergenz*, zwingt den latenten Raum nah an eine Standardnormalverteilung $\mathcal{N}(0, I)$ zu bleiben, sodass neue Daten durch einfaches Sampling aus $p(z)$ generiert werden können.

5.3 Diffusion Model

Ein *Diffusion Model* oder auch *Diffusions-Wahrscheinlichkeitsmodell* ist ein generatives Modell, das zur Klasse der latenten Variablenmodelle gehört [21, S. 153] [18, S. 114].

Das grundlegende Prinzip besteht darin, einen Diffusionsprozess umzukehren, bei dem schrittweise Rauschen zu den Daten hinzugefügt wird. Durch die Umkehrung dieses Prozesses kann das Modell aus verrauschten Signalen Daten rekonstruieren und so neue Daten generieren. Dafür wird eine parametrisierte Markov-Kette mittels Variationsinferenz trainiert, deren Übergänge lernen, den Diffusionsprozess umzukehren [21, S. 153] [22, S. 2].

5.3.1 Markov-Ketten

Eine Markov-Kette ist ein stochastischer Prozess, bei dem der zukünftige Zustand ausschließlich vom aktuellen Zustand abhängt und nicht von der Vergangenheit. Diese Eigenschaft wird als *Gedächtnislosigkeit* bezeichnet. Für eine Sequenz von Zufallsvariablen $x_0, x_1, \dots, x_T$ lässt sich dies ausdrücken als:

$$p(x_t | x_0, \dots, x_{t-1}) = p(x_t | x_{t-1}), \quad t = 1, \dots, T \quad (5.3)$$

Hierbei bezeichnet t den Zeitschritt und x_t den Zustand zum Zeitpunkt t . Eine solche Markov-Kette, bei der der aktuelle Zustand nur vom unmittelbar vorhergehenden abhängt, wird als Markov-Kette *erster Ordnung* bezeichnet [17, S. 609]. In Abb. 5.1 wird diese graphisch dargestellt. Aufgrund der Markov-Eigenschaft kann die gemeinsame Wahr-

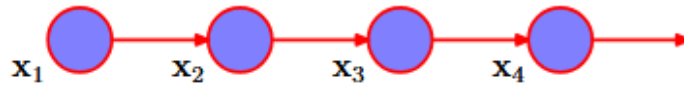


Abb. 5.1: Markov-Kette erster Ordnung [17, S. 609]

scheinlichkeit der gesamten Sequenz $x_{1:T}$ gegeben x_0 als Produkt der einzelnen Übergangswahrscheinlichkeiten faktorisiert werden:

$$p(x_{1:T} | x_0) = \prod_{t=1}^T p(x_t | x_{t-1}) \quad (5.4)$$

Diese Faktorisierung ist für Diffusionsmodelle von zentraler Bedeutung [21, S. 153 f.] [17, S. 608 f.]. Die Gedächtnislosigkeit der Markov-Kette erweist sich als besonders geeignet für Diffusionsprozesse, da jeder Schritt sowohl beim Hinzufügen als auch beim Entfernen von Rauschen ausschließlich vom aktuellen Zustand abhängt.

5.3.2 Noise-Schedule

Der *Noise Schedule* definiert eine Sequenz von vorgegebene Werten $\beta_1, \beta_2, \dots, \beta_T$ mit $\beta_t \in (0,1)$, die steuern wie viel Rauschen in jedem Schritt t hinzugefügt wird. Aus diesen werden zwei Werte definiert [19, S. 43 f.] [21, S. 155 f.]:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{k=1}^t \alpha_k \quad (5.5)$$

Dabei gibt α_t an, welcher Anteil des Signals aus dem vorherigen Schritt erhalten bleibt. Hingegen tut $\bar{\alpha}_t$ das kumulative Produkt über alle bisherigen Schritte darstellen. Mit steigendem t nimmt $\bar{\alpha}_t$ monoton ab, sodass das ursprüngliche Signal zunehmend durch Rauschen überschrieben wird [19, S. 44]. Heißt, wenn für $t \rightarrow T$ gilt $\bar{\alpha}_T \approx 0$, was bedeutet, dass das Originalbild nahezu vollständig zerstört ist.

Die Wahl des Noise Schedulers beeinflusst die Qualität der generierten Daten. Der *linear Schedule*, der von Ho et al. [22, S. 5] verwendet wird, definiert β_t als lineare Interpolation zwischen einem Start- und Endwert:

$$\beta_t = \beta_1 + \frac{t-1}{T-1}(\beta_T - \beta_1) \quad (5.6)$$

Allerdings wurde beobachtet dass der lineare Schedule in den späten Schritten zu aggressiv rauscht. Nichol und Dhariwal [23, S. 4] verwenden daher einen *Cosine Schedule*, der $\bar{\alpha}_t$ direkt über eine Kosinusfunktion definiert:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad \text{mit} \quad f(t) = \cos\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)^2 \quad (5.7)$$

wobei s ein kleiner Offset-Parameter ist, der verhindert, dass β_t in der Nähe von $t = 0$ zu klein wird. Der Cosine Schedule führt zu einem gleichmäßigeren Rauschverlauf und verbessert die Bildqualität insbesondere bei niedrigen Auflösungen.

TODO: Vlt. Bild hinzufügen aus Nichol?

5.3.3 Vorwärtsprozess

Die Abb. 5.2 stellt den Vorwärts- und Rückwärtsprozess an einem Bild dar. Der Vorwärts-

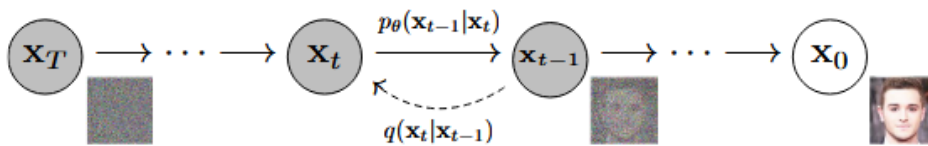


Abb. 5.2: Darstellung der Vorwärts- und Rückwärtsprozesse

prozess (engl. *Forward Process*) ist fest vorgegeben, wird nicht trainiert und dient als Encoder. Er fügt einem Signal x_0 schrittweise Gaußsches Rauschen hinzu bis am Ende reines Rauschen übrig ist. Die Übergangsverteilung für einen einzelnen Schritt ist definiert als [21, S. 155 f.]:

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right) \quad (5.8)$$

Hierbei wird das Signal aus dem vorherigen Schritt mit dem Faktor $\sqrt{1 - \beta_t}$ skaliert und Gaußsches Rauschen mit Varianz β_t hinzugefügt. Wobei eben $0 < \beta_1 < \beta_T < 1$ gilt. Da jeder Schritt nur vom vorherigen Zustand abhängt, kann die gemeinsame Verteilung des

gesamten Vorwärtsprozesses gemäß der Markov-Eigenschaft faktorisiert werden mit [21, S. 156] [22, S. 2]:

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}) \quad (5.9)$$

Dies bedeutet, dass ein Bild x_0 nach T Schritten vollständig zerstört wird und das Ergebnis x_T reinem Gaußschem Rauschen $\mathcal{N}(0, I)$ entspricht. Ein Vorteil des Vorwärtsprozesses ist, dass x_t nicht iterativ berechnet werden muss. Mithilfe der Definitionen $\alpha_t = 1 - \beta_t$ und $\tilde{\alpha}_t = \prod_{k=1}^t \alpha_k$ lässt sich x_t direkt aus dem Originalbild x_0 berechnen [21, S. 156] [22, S. 2] [19, S. 44 f.]:

$$q(x_t | x_0) = \mathcal{N}\left(x_t; \sqrt{\tilde{\alpha}_t} x_0, (1 - \tilde{\alpha}_t) I\right) \quad (5.10)$$

Daraus ergibt sich die folgende Formel [21, S. 156] [22, S. 2] [19, S. 44 f.]:

$$x_t = \sqrt{\tilde{\alpha}_t} x_0 + \sqrt{1 - \tilde{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (5.11)$$

Dadurch kann direkt zu einem beliebigen Rauschzustand x_t gesprungen werden ohne alle Zwischenschritte $x_0, x_1, \dots, x_{t-1}$ durchlaufen zu müssen. Dies ermöglicht dann ein effizientes, da zufällige Zeitschritte t gesampelt werden können [22, S. 2 f.].

5.3.4 Rückwärtsprozess

Der Rückwärtsprozess (engl. *Reverse Process*) ist das Gegenstück zum Vorwärtsprozess und dient als lernbarer Decoder. Er startet bei reinem Rauschen $x_T \sim \mathcal{N}(0, I)$ und entfernt in jedem Schritt einen Teil des Rauschens, bis ein sinnvolles Datensample x_0 rekonstruiert ist. Auch dieser Prozess wird als Markov-Kette dargestellt [22, S. 2]:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t) \quad (5.12)$$

Da die wahren Übergangswahrscheinlichkeiten $q(x_{t-1} | x_t)$ von der unbekannten Datenverteilung abhängen, wird jeder Rückwärtsschritt durch ein neuronales Netz mit Parametern θ approximiert [22, S. 2]:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}\left(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I\right) \quad (5.13)$$

Parametrisierung des Mittelwerts

Anstatt den Mittelwert μ_θ direkt zu schätzen, wird das Netzwerk darauf trainiert, das hinzugefügte Rauschen $\epsilon_\theta(x_t, t)$ vorherzusagen. Aus der Gl. (5.11) des Vorwärtsprozesses lässt sich der Mittelwert dann wie folgt berechnen [22, S. 4]

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \tilde{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \quad (5.14)$$

Diese Parametrisierung ist vorteilhaft, da die Vorhersage des Rauschens stabiler zu optimieren ist als die direkte Mittelwertschätzung. Damit ergibt sich der vollständige Sampling-Schritt im Rückwärtsprozess als [22, S. 4]:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I) \quad (5.15)$$

Die Varianz σ_t^2 wird üblicherweise als fester Wert gesetzt, wobei $\sigma_t^2 = \beta_t$ oder $\sigma_t^2 = \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t$ verwendet wird und für z entweder $z \sim \mathcal{N}(0, I)$ oder $z = 0$ gilt, wenn es für den letzten Schritt $t = 1$ gilt [22, S. 4].

5.3.5 Sampling

Das Sampling beschreibt den Prozess der Datengenerierung, bei dem von reinem Rauschen $x_T \sim \mathcal{N}(0, I)$ schrittweise der Rückwärtsprozess durchlaufen wird [22, S. 4]. Die Wahl des Sampling-Verfahrens beeinflusst sowohl die Qualität der generierten Daten als auch die benötigte Rechenzeit.

DDPM-Sampling

Sofern nicht anders angegeben, beziehen sich die folgenden Aussagen auf Ho, Jain und Abbeel [22, S. 3 ff.]. Das Diffusion Denoising Probabilistic Model (DDPM)-Sampling implementiert den stochastischen Rückwärtsprozess aus Gl. (5.15) und entspricht damit der ursprünglichen Sampling-Prozedur von Ho, Jain und Abbeel. Ausgehend von reinem Gaußschen Rauschen $x_T \sim \mathcal{N}(0, I)$ wird in jedem Schritt $t \rightarrow t-1$ eine kleine Menge des im Bild enthaltenen Rauschens entfernt, wobei das trainierte Netzwerk $\epsilon_\theta(x_t, t)$ diese Rauschkomponente vorhersagt. Der erste Klammerausdruck aus Gl. (5.15) entspricht dem geschätzten Mittelwert der Rückwärtsverteilung $p_\theta(x_{t-1} | x_t)$, während über das hinzugefügte Rauschen $\sigma_t z$ mit $z \sim \mathcal{N}(0, I)$ die für den Markov-Prozess charakteristische Stochastizität eingebracht wird. Für die Varianz wird typischerweise $\sigma_t^2 = \beta_t$ gewählt. Der Prozess durchläuft alle T Zeitschritte, wobei in der Praxis $T = 1000$ mit einem linearen β -Schedule von $\beta_1 = 10^{-4}$ bis $\beta_T = 0,02$ verwendet wird. Da bei jedem dieser Schritte eine vollständige Auswertung des Neuronalen Netzes erforderlich ist, liefert das Verfahren zwar qualitativ hochwertige Ergebnisse, ist jedoch rechenintensiv und in der Praxis deutlich langsamer als alternative generative Modelle [vgl. 24, S. 3].

DDIM-Sampling

Sofern nicht anders angegeben, beziehen sich die folgenden Aussagen auf Song, Meng und Ermon [24, S. 3 ff.]. Im Gegensatz zum DDPM-Sampling müssen beim Diffusion Denoising Implicit Model (DDIM)-Sampling nicht alle T Schritte durchlaufen werden. Dies wird dadurch ermöglicht, dass der Vorwärtsprozess auch als nicht-Markov-Prozess in-

terpretiert werden kann, was zu einer deterministischen Sampling-Prozedur führt. Das DDIM-Sampling wird wie folgt beschrieben:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(x_t, t) + \sigma_t z \quad (5.16)$$

Der erste Term entspricht der Vorhersage des sauberen Bildes x_0 ausgehend vom ver-rauschten Zustand x_t , der zweite Term stellt die deterministische Richtung dar, die zu-rück auf x_{t-1} zeigt, und der letzte Term $\sigma_t z$ mit $z \sim \mathcal{N}(0, I)$ steuert über σ_t den Grad der Stochastizität des Verfahrens. Für $\sigma_t = 0$ wird der Prozess deterministisch, das heißt der-selbe Startpunkt x_T erzeugt immer dasselbe Ergebnis. In der Praxis reichen häufig 50 bis 100 Schritte statt $T = 1000$, was eine 10- bis 50-fache Beschleunigung bei vergleichbarer Bildqualität ermöglicht [24, S. 7].

DPM-Solver++

DPM-Solver++ ist ein numerischer Löser für die zum Diffusionsprozess gehörende ge-wöhnliche Differentialgleichung (Diffusion Ordinary Differential Equation (ODE)) und stellt eine Weiterentwicklung des ursprünglichen DPM-Solvers dar [vgl. 25, S. 2]. Grun-didee dieses Verfahrens ist es, den Sampling-Prozess als Lösen dieser ODE zu interpre-tieren und durch eine Variablentransformation auf das log-SNR $\lambda_t = \log(\alpha_t/\sigma_t)$ den li-nearen Anteil der Lösung analytisch zu berechnen. Lediglich der nichtlineare, durch das Neuronale Netz parametrisierte Anteil muss noch durch eine Taylor-Entwicklung ap-proximiert werden [vgl. 25, S. 4 f.]. Im Gegensatz zum ursprünglichen DPM-Solver, der hierfür das Rauschvorhersagemodell ϵ_θ verwendet, basiert DPM-Solver++ auf dem Da-tenvorhersagemodell x_θ , welches direkt das saubere Bild x_0 schätzt. Dies reduziert den Diskretisierungsfehler bei großen Guidance-Skalen und erlaubt zudem den Einsatz von Thresholding-Methoden, die den vorhergesagten Wert auf den gültigen Datenbereich be-schränken [vgl. 25, S. 5 ff.].

In der Praxis kommt überwiegend die Multistep-Variante zweiter Ordnung DPM-Solver++(2M) zum Einsatz. Ein Solver zweiter Ordnung berücksichtigt in der Taylor-Entwicklung zusätzlich die erste Ableitung des Vorhersagemodells und erreicht so einen geringeren Diskretisierungsfehler pro Schritt als ein Solver erster Ordnung. Die benötigte Ableitung wird dabei aus zwischengespeicherten Modellauswertungen vorheriger Schrit-te approximiert, sodass pro Iteration nur eine einzige Auswertung des Neuronalen Netzes erforderlich ist [vgl. 25, S. 6 f.]. Mit der Schrittweite $h_i := \lambda_{t_i} - \lambda_{t_{i-1}}$ und dem Verhältnis $r_i := h_{i-1}/h_i$ ergibt sich der Update-Schritt zu:

$$\tilde{x}_{t_i} = \frac{\sigma_{t_i}}{\sigma_{t_{i-1}}} \tilde{x}_{t_{i-1}} - \alpha_{t_i} (e^{-h_i} - 1) D_i, \quad D_i = \left(1 + \frac{1}{2r_i} \right) x_\theta(\tilde{x}_{t_{i-1}}, t_{i-1}) - \frac{1}{2r_i} x_\theta(\tilde{x}_{t_{i-2}}, t_{i-2}) \quad (5.17)$$

Der erste Iterationsschritt wird durch eine Aktualisierung erster Ordnung initialisiert, die dem deterministischen DDIM-Sampling entspricht [vgl. 25, S. 9]. Aus der Forschung er-zeugt DPM-Solver++(2M) bereits in 15 bis 20 Schritten Bilder vergleichbarer Qualität wie ein DDIM-Sampling mit mehreren hundert Schritten [vgl. 25, S. 11].

UniPC

Sofern nicht anders angegeben, beziehen sich die folgenden Aussagen auf Zhao u. a. [26, S. 1 ff.]. Unified Predictor-Corrector (UniPC) ist ein trainingsfreies Sampling-Verfahren, das insbesondere bei sehr wenigen Schritten (< 10) eine bessere Sampling-Qualität als vergleichbare Verfahren erreicht. Die Grundidee besteht darin, jeden Sampling-Schritt in zwei Teilschritte aufzuteilen. Zuerst erzeugt der Prädiktor (UniP) analog zum DPM-Solver++ eine erste Schätzung des nachfolgenden Zustands $\tilde{x}_{t_i}$. Anschließend verbessert der Korrektor (UniC) diese Schätzung zu einem korrigierten Wert $\tilde{x}_{t_i}^c$, indem die Modellauswertung am vorhergesagten Punkt als zusätzlicher Stützpunkt einbezogen wird. Dadurch erhöht sich die Genauigkeitsordnung des Verfahrens um eins. Der wesentliche Unterschied zu klassischen Predictor-Corrector-Verfahren aus der Mathematik liegt darin, dass UniC keine zusätzlichen Auswertungen des Neuronalen Netzes benötigt. Die im Korrekturschritt berechnete Modellauswertung wird im darauffolgenden Sampling-Schritt wiederverwendet, sodass pro Iteration nur eine einzige Netzauswertung erforderlich ist.

Wie der DPM-Solver++ basiert UniPC auf der Lösung der Diffusion-ODE im log-SNR-Bereich. Mit der Schrittweite $h_i := \lambda_{t_i} - \lambda_{t_{i-1}}$ und den Differenzen D_m aus zusätzlichen Modellauswertungen ergibt sich der Korrekturschritt UniC-p zu:

$$\tilde{x}_{t_i}^c = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}}^c - \sigma_{t_i} (e^{h_i} - 1) \epsilon_{\theta}(\tilde{x}_{t_{i-1}}, t_{i-1}) - \sigma_{t_i} B(h_i) \sum_{m=1}^p \frac{a_m}{r_m} D_m \quad (5.18)$$

Die ersten beiden Terme entsprechen einer Aktualisierung erster Ordnung, die dem deterministischen DDIM-Sampling entspricht. Der Summenterm bringt die Korrektur höherer Ordnung ein, indem die Differenzen D_m vorangegangener Modellauswertungen mit den Koeffizienten a_m gewichtet kombiniert werden. Der Prädiktor UniP-p ergibt sich aus Gl. (5.18) durch Weglassen des Terms D_p , da dieser die im Prädiktorschritt noch nicht verfügbare Modellauswertung am aktuellen Zeitschritt voraussetzt. Die Kombination beider Komponenten erreicht eine Genauigkeitsordnung von $p + 1$. Da Prädiktor und Korrektor derselben analytischen Form folgen, lassen sich beliebige Ordnungen einheitlich formulieren, während der DPM-Solver++ nur bis zur dritten Ordnung explizit angegeben werden kann.

5.4 Grundlegende Architekturen

5.4.1 Convolutional Neural Network

Convolutional Neural Network (CNN) ist eine Klasse neuronaler Netzwerke, die speziell für die Verarbeitung von Bilddaten entwickelt wurde [27, S. 1]. Dies geschieht durch sogenannte Faltungsschichten (engl. *convolutional layers*), die mithilfe von Filtern lokale Muster in der Datenstruktur erkennen. Diese Merkmalsextraktion ermöglicht es dem Modell, komplexe Strukturen mit hoher Genauigkeit zu identifizieren [28, S. 1170].

Die Faltung (engl. *convolution*) ist eine mathematische Operation, bei der ein Faltungskern (engl. *kernel*) schrittweise über ein Bild geschoben wird. An jeder Position wird das

elementweise Produkt zwischen Kernel und dem überdeckten Bildausschnitt aufsummiert. Formal ergibt sich für ein zweidimensionales Bild f mit Kernel k :

$$g(x, y) = \sum_i \sum_j f(x + i, y + j) \cdot k(i, j) \quad (5.19)$$

Das Ergebnis ist eine *Feature Map*, die beschreibt, wie stark ein bestimmtes Muster, wie eine Kante oder Textur an jeder Bildposition vorhanden ist [29, S. 14]. In Abb. 5.4 wird ein

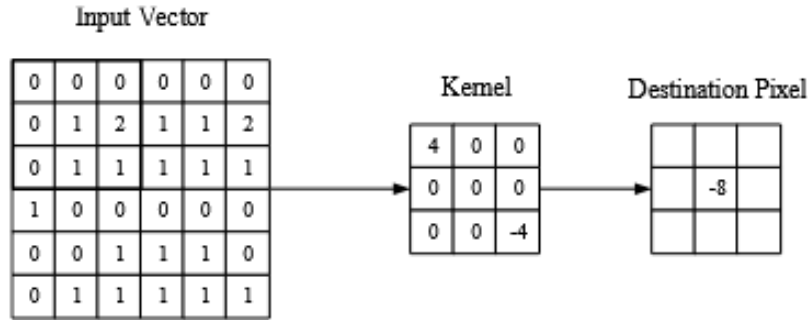


Abb. 5.3: Convolutional [27, S. 6]

CNN dargestellt. Die Architektur basiert auf drei Schichttypen. In der Faltungsschicht, wird wie der Name es bezeichnet die Faltung durchgeführt und Merkmale extrahiert [27, S. 5 f.]. Parallel zu den Faltungsoperationen folgt eine Aktivierungsfunktion (ReLU). Diese führt eine Nichtlinearität hinzu und reduziert das Vanishing-Gradient-Problem [28, S. 1169]. Als Aktivierungsfunktion wird standardmäßig die Rectified Linear Unit (ReLU) mit $f(x) = \max(0, x)$ eingesetzt, die ein schnelleres Training als traditionelle Funktionen wie $\tanh(x)$ ermöglicht [30, S. 3].

Darauf folgt die *Pooling-Schicht*. Diese reduziert die räumliche Auflösung, typischerweise durch Max-Pooling mit 2×2 -Fenstern, wodurch die Aktivierungskarte auf 25% ihrer Größe verkleinert wird. Damit werden die Anzahl der Parameter sowie die Rechenkomplexität des Modells schrittweise reduziert [27, S. 8].

Die *Fully-Connected-Schicht* am Ende des Netzwerks verbindet jeden Neuron direkt mit allen Neuronen der benachbarten Schichten, analog zur klassischen ANN-Architektur. Dadurch werden die extrahierten Merkmale zu Klassenwahrscheinlichkeiten verarbeitet [27, S. 8]. Im Gegensatz zu traditionellen ANN, bei denen jedes Neuron mit allen Neuronen der vorherigen Schicht verbunden ist, verbinden sich Neuronen in CNN nur mit einem kleinen lokalen Bereich der Eingabe [27, S. 6]. Diese lokale Konnektivität reduziert die Anzahl der Parameter erheblich und macht das Training effizienter [27, S. 2–3].

Eine typische CNN-Architektur stapelt mehrere Faltungs- und Pooling-Schichten, wobei frühe Schichten einfache Merkmale und tiefere Schichten zunehmend komplexe, abstrakte Repräsentationen lernen [27, S. 8–9]. Diese hierarchische Merkmalsextraktion bildet die Grundlage für den Encoder-Pfad in Architekturen wie U-Net [31, S. 6].

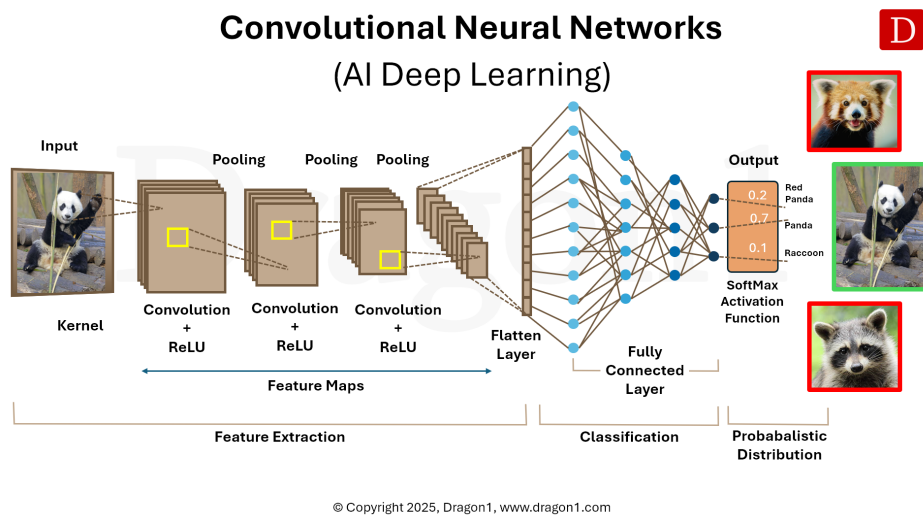


Abb. 5.4: CNN

5.4.2 U-Net

U-Net ist eine CNN-Architektur, die für die biomedizinische Bildsegmentierung entwickelt wurde. Trotz ihres ursprünglichen Anwendungsbereiches dient die Architektur eine zentrale Rolle im Denoising-Schritt in Diffusionsmodelle [32, S. 2]. In Abb. 5.5 wird

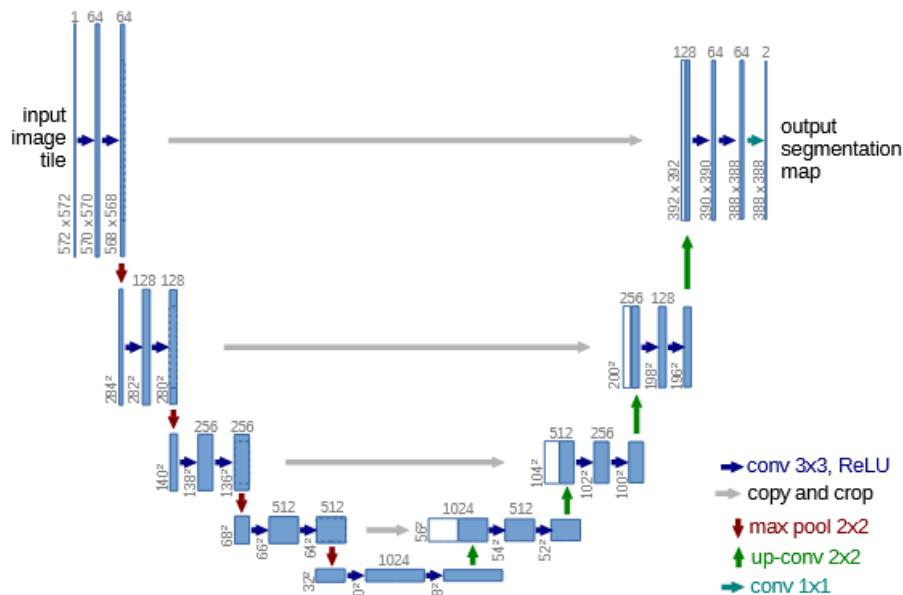


Abb. 5.5: U-Net Architektur [32, S. 2]

die Architektur dargestellt. Das Grundprinzip von U-Net basiert auf einer symmetrischen Encoder-Decoder-Struktur. Der Encoder, auch als *Contracting Path* bezeichnet, folgt dem Prinzip eines typischen CNNs und besteht aus wiederholten Blöcken von zwei 3×3-Faltungen (unpadded) mit anschließender ReLU-Aktivierung sowie einem 2×2 Max-

Pooling mit Schrittweite 2 zum Downsampling [32, S. 4]. Mit jeder Downsampling-Stufe verdoppelt sich die Anzahl der Feature-Kanäle, während die räumliche Auflösung schrittweise reduziert wird [32, S. 4]. Dieser Prozess ermöglicht es dem Netzwerk, zunehmend abstrakte semantische Merkmale zu extrahieren und hierarchische Repräsentationen zu erlernen [31, S. 6].

Der Decoder, auch *Expansive Path* genannt, spiegelt diese Struktur symmetrisch und besteht aus Upsampling-Schritten mittels 2×2 -Up-Convolutions, gefolgt von zwei 3×3 -Faltungen mit ReLU [32, S. 4]. Das zentrale Merkmal der U-Net-Architektur sind die sogenannten Skip Connections, dargestellt als graue Pfeile in Abb. 5.5. Diese verbinden korrespondierende Encoder- und Decoder-Ebenen direkt miteinander, indem die Feature-Maps konkateniert werden [32, S. 4] [31, S. 9]. Dadurch können feinkörnige räumliche Informationen, die beim Downsampling verloren gehen, im Decoder wiederhergestellt werden. Die finale Schicht verwendet eine 1×1 -Faltung, um die Feature-Vektoren auf die gewünschte Anzahl von Klassen abzubilden [32, S. 4].

Mathematisch lässt sich ein einzelner Faltungsschritt beschreiben als:

$$\mathbf{x}^{(l+1)} = \sigma\left(W^{(l)} * \mathbf{x}^{(l)} + b^{(l)}\right), \quad (5.20)$$

wobei $W^{(l)}$ den Faltungsfilter, $\mathbf{x}^{(l)}$ die Eingabe-Feature-Map, $b^{(l)}$ den Bias und σ die ReLU-Aktivierungsfunktion der l -ten Schicht bezeichnen. Die Skip Connection im Decoder lässt sich formal als $\mathcal{F}_x^{(l)} = \text{Concat}(\mathcal{E}_x^{(l)}, \mathcal{D}_x^{(l)})$ beschreiben, wobei $\mathcal{E}_x^{(l)}$ und $\mathcal{D}_x^{(l)}$ die Feature-Maps von Encoder und Decoder auf Ebene l repräsentieren und $\mathcal{F}_x^{(l)}$ die fusionierte Feature-Map darstellt [31, S. 9]. Diese Konkatenation ermöglicht es, hochauflösende strukturelle Details aus dem Encoder mit den abstrakten semantischen Repräsentationen des Decoders zu verbinden.

Im Kontext von Diffusionsmodellen wird U-Net eingesetzt, um bei jedem Denoising-Schritt das Rauschen $\epsilon_\theta(\mathbf{x}_t, t)$ zu schätzen, wobei der Zeitschritt t als zusätzliche Konditionierungsinformation in die Architektur einfließt. Die Vielseitigkeit von U-Net zeigt sich nicht nur in der Anwendung über verschiedene Domänen hinweg, sondern auch in der Integration in unterschiedliche Modelltypen wie Diffusionsmodelle, Generative Adversarial Networks (GANs) und autoregressive Architekturen. Durch die Flexibilität der symmetrischen Encoder-Decoder-Struktur mit Skip Connections hat sich U-Net als Standardarchitektur in modernen Diffusionsmodellen für verschiedene Datenmodalitäten etabliert, darunter Bild-, Audio-, Video- und 3D-Generierung [31, S. 2].

Die U-Net-Architektur bietet mehrere Vorteile für generative Aufgaben. Die hierarchische Encoder-Decoder-Struktur ermöglicht eine Merkmalsextraktion auf mehreren Auflösungsebenen, wodurch sowohl globaler Kontext als auch lokale Details erfasst werden können. Die Skip Connections behalten detaillierte räumliche, temporale und strukturelle Details, die bei herkömmlichen Autoencoder-Architekturen durch das Downsampling verloren gehen würden. Darüber hinaus besitzt U-Net eine hohe Recheneffizienz, da die faltungsbasierten Operationen parallelisiert werden können und im Gegensatz zu Transformer-Modellen linear mit der Eingabegröße skalieren. Weitere Stärken umfassen die Robustheit gegenüber verrauschten oder unvollständigen Eingabedaten sowie die Flexibilität zur Integration von Attention-Mechanismen und Residual Connections [31, S. 34 ff.].

Trotz dieser Vorteile weist U-Net auch Limitationen auf. Eine wesentliche Einschränkung ist die begrenzte Fähigkeit von globalem Kontext, da die faltungsbasierte Struktur primär auf lokale räumliche Merkmale spezialisiert ist. Bei hochauflösenden generativen Aufgaben steigt der Speicherbedarf erheblich, während das rezeptive Feld der Faltungsschichten proportional kleiner wird. Zudem kann die Generalisierungsfähigkeit bei domänenfremden Daten oder multimodalen Aufgaben wie Text-zu-Bild-Generierung eingeschränkt sein, da U-Net auf pixelbasierte Transformationen ausgelegt ist. Schließlich operiert U-Net wie viele tiefe neuronale Architekturen als Black-Box, was die Interpretierbarkeit der gelernten Repräsentationen erschwert [31, S. 32 f.] [31, S. 40 f.].

5.5 Einsatzmöglichkeiten in der Medizin

TODO: Benötigt?

6 Datenqualität und ethische Aspekte

Das Konzept der Generierung synthetischer Daten lässt sich auf die Arbeiten von Metropolis und Ulam im Jahr 1949 zurückführen, die mit der Entwicklung der Monte-Carlo-Simulationsmethoden den Grundstein für die rechnergestützte Erzeugung künstlicher Datenpunkte legten [33, S. 335 f.]. Seitdem haben sich die Methoden zur synthetischen Datenerzeugung weiterentwickelt und finden heute in verschiedenen Anwendungsbereichen Einsatz, wovon eines davon in der Medizin ist. In diesem Kapitel werden die Aspekte der Datenqualität sowie die ethischen Herausforderungen behandelt, die mit der Nutzung synthetischer Daten einhergehen.

6.1 Definition synthetischer Daten

Bevor auf die Erzeugung und Anwendung synthetischer Daten eingegangen wird, ist zunächst eine Definition nötig, um zu verstehen was synthetische Daten sind. Jordon u. a. von der Royal Society und dem Alan Turing Institute definieren den Begriff wie folgt:

“Synthetic data is data that has been generated using a purpose-built mathematical model or algorithm, with the aim of solving a (set of) data science task(s).” [34, S. 5]

Aus dieser Definition lassen sich zwei Aspekte ableiten: Zum einen werden synthetische Daten nicht zufällig erzeugt, sondern gezielt durch mathematische Modelle oder Algorithmen generiert. Darunter fallen beispielsweise GAN, VAE oder Diffusionsmodelle, die jeweils unterschiedliche Strategien zur Datenerzeugung verfolgen. Zum anderen sind synthetische Daten immer zweckgebunden. Das heißt, sie werden gezielt eingesetzt, um datengetriebene Aufgaben wie Klassifikation, Vorhersage oder den Schutz sensibler Informationen zu lösen.

Unabhängig von der Erzeugungsmethode lassen sich synthetische Daten in partiell und vollständig synthetische Daten unterteilen [vgl. 35, S. 2]. Partiiell synthetische Daten kombinieren reale Datensätze mit synthetisch erzeugten Elementen, wobei gezielt Variablen mit hohem Offenlegungsrisiko ersetzt werden, während die restlichen Merkmale erhalten bleiben. Im medizinischen Bereich wird dieser Ansatz etwa genutzt, um die Privatsphäre von Patientinnen und Patienten zu schützen, um dennoch aussagekräftige Analysen zu erlauben [vgl. 36, S. 7]. Vollständig synthetische Daten hingegen werden komplett durch statistische Modelle erzeugt, ohne dass die Originaldaten preisgegeben werden. Dabei kommt das Verfahren der multiplen Imputation zum Einsatz, bei dem alle nicht erhobenen Werte als fehlend betrachtet und wiederholt durch plausible Werte ersetzt werden [vgl. 37, S. 2]. Beide Ansätze können dazu beitragen, Probleme wie Datenknappheit oder Datenschutzerfordernngen anzugehen. Mehr dazu in Abschnitt 6.2.

6.2 Ethische Aspekte synthetischer medizinischer Daten

Obwohl synthetische Daten von rechnergestützten Systemen generiert werden und keine unmittelbaren Patienteninformationen enthalten, ergeben sich durch ihre Erzeugung und Nutzung im medizinischen Umfeld ethische Herausforderungen. Da generative Modelle auf realen Patientendaten trainiert werden, können sich Probleme des Datenschutzes und der Verantwortung auch auf den synthetischen Datenraum übertragen.

Medizinische Bilddaten zählen zu den schützenswerten personenbezogenen Daten. Art. 9 Abs. 1 DSGVO legt fest, dass die Verarbeitung von „Gesundheitsdaten [...] einer natürlichen Person [...] untersagt“ ist. Von diesem Verarbeitungsverbot darf nur unter engen Voraussetzungen, wie einer ausdrücklichen Einwilligung ausgewichen werden [38]. Dies schränkt die Entwicklung KI-basierter Verfahren in der medizinischen Bildverarbeitung ein, da der Zugang zu Trainingsdaten begrenzt ist. Synthetische Daten bieten einen Lösungsansatz, da sie keine direkten Bezüge zu realen Personen enthalten und daher außerhalb des Anwendungsbereichs der DSGVO liegen.

Ergänzend stellt die KI-Verordnung Anforderungen an die Datenbasis KI-basierter Anwendungen in der Medizin, die nach Art. 6 Abs. 1 AI Act als Hochrisiko-KI-Systeme gelten [39]. So fordert Art. 10 AI Act, dass Trainings-, Validierungs- und Testdatensätze „relevant, hinreichend repräsentativ und so weit wie möglich fehlerfrei und vollständig“ sind [40]. Besonders die Forderung nach Repräsentativität ist aus ethischer Sicht von Bedeutung, da sie für eine gerechte und diskriminierungsfreie Anwendung medizinischer KI-Systeme bildet. Hieraus ergibt sich das Problem, dass die Verzerrungen (auch *Bias* genannt) in den Daten vermieden werden müssen. Denn es besteht bei der Generierung synthetischer Daten das Risiko, dass identifizierbare Merkmale erhalten bleiben und sensible Informationen indirekt preisgegeben werden. Zwar können sensible Daten vor dem Training anonymisiert werden, eine zu starke Anonymisierung verringert jedoch den analytischen Wert der Daten [vgl. 41, S. 192]. Somit besteht ein Kompromiss zwischen Anonymität und Wert der Daten.

6.3 Risiken und Grenzen synthetischer Daten

Neben den ethischen Fragestellungen weisen synthetische Daten auch technische und methodische Grenzen auf. Eine Herausforderung liegt in der medizinischen Plausibilität synthetischer Daten. Generative Modelle können visuell gute Ergebnisse liefern, dabei jedoch anatomisch fehlerhafte oder medizinisch unrealistische Strukturen erzeugen. Diese Daten werden als Halluzinationen bezeichnet. Diese Halluzinationen führen dazu, dass die synthetischen Bilder bei der Auswertung irreführende Ergebnisse liefern und somit die Zuverlässigkeit des KI-Modells verringert [vgl. 42, S. 10]. Hinzu kommt, dass GANs häufig unter Mode Collapse leiden, wodurch die Vielfalt der generierten Bilder eingeschränkt wird. Jedoch wird dieses Problem durch die Verwendung von beispielsweise Diffusions Modellen verringert [vgl. 42, S. 5] [vgl. 43, S. 1].

Dazu zählt aber jedoch das Problem der Generalisierbarkeit. Modelle für medizinische Bilddaten lernen die Muster ihrer Trainingsdaten. Jedoch kann diese Fähigkeit aber nicht zwangsläufig auf neue, abweichende Bildmuster übertragen. Beispielsweise ein

Klassifikationsmodell für die Hautläsion, das überwiegend mit Bildern heller Hauttypen trainiert wurde, liefert bei dunkleren Hauttypen schlechtere Ergebnisse [vgl. 44, S. 18]. Da generative Modelle die Eigenschaften ihrer Trainingsdaten übernehmen, werden solche Verzerrungen auch in synthetischen Daten weitergegeben. Hinzu kommt, dass generative Modelle für ihr initiales Training selbst auf reale klinische Bilddaten angewiesen sind [vgl. 42, S. 10]. Synthetische Daten bieten somit nur eine Teillösung für den Mangel an medizinischen Trainingsdaten und können reale Patientendaten nicht komplett ersetzen.

6.4 Qualitätskriterien für synthetische medizinische Bilddaten

Um die Eignung synthetischer Bilddaten für medizinische Anwendungen beurteilen zu können, sind verlässliche Qualitätskriterien erforderlich. Diese lassen sich in visuelle und mathematische Bewertungsverfahren unterteilen, die im Folgenden näher beschrieben werden.

6.4.1 Visuelle Bewertungskriterien

Die visuelle Bewertung synthetischer medizinischer Bilder erfolgt im besten Fall durch medizinische Fachkräfte. Diese Form der Beurteilung ist deshalb relevant, weil rein mathematische Metriken medizinisches Fachwissen nicht abbilden können und daher nicht zur Beurteilung der medizinischen Plausibilität geeignet sind [vgl. 45, S. 6] [vgl. 46, S. 6].

In einer Studie von Schuit, Parra und Besa wurde eine solche Reader Study mit drei Radiologen durchgeführt. Die Teilnehmer sollten sowohl zwischen realen und synthetischen Röntgenaufnahmen unterscheiden als auch die Konsistenz zwischen Bildmerkmalen und Zielpathologie bewerten. Die Ergebnisse zeigen, dass insbesondere Diffusionsmodelle visuell realistische Röntgenaufnahmen erzeugen können [vgl. 46, S. 1].

Dennoch weist die visuelle Bewertung Einschränkungen auf. Sie subjektiv und von der Erfahrung der beurteilenden Person abhängig. Zudem lässt sie sich schwer auf große Datenmengen skalieren, weshalb die Bewertung durch mathematische Metriken ergänzt werden kann.

6.4.2 Mathematische Bewertungsmetriken

Um die Qualität von durch generativen Modelle erstellten Bildern objektiv zu beurteilen, kommen sogenannte Image Quality Metrics (IQMs) zum Einsatz. IQMs erlauben einen objektiven Rahmen zur Bewertung von Eigenschaften wie Auflösung, Rauschen und Artefakte. Gegenüber subjektiven Bewertungen durch Menschen haben IQMs den Vorteil, dass sie standardisiert, quantifizierbar und reproduzierbar sind und somit einen systematischen Vergleich verschiedener generativer Modelle ermöglicht. IQMs lassen sich in drei verschiedenen Kategorien einordnen: full-reference (FR), reduced-reference (RR) und

no-reference (NR). Diese Unterscheidung ist im Kontext von Diffusionsmodellen besonders wichtig. Bei Aufgaben wie Bildrekonstruktion oder Denoising, bei denen ein Originalbild als Referenz vorliegt, kommen FR-Metriken wie SSIM oder Peak Signal-to-Noise Ratio (PSNR) zum Einsatz. Bei der freien Text-to-Image-Generierung hingegen existiert kein eindeutiges Referenzbild, weshalb hier häufig auf NR-Metriken oder semantische Maße wie den FID oder den CLIP-Score zurückgegriffen wird [47, S. 1]. **TODO: mehr quelle zu IQM finden**

Fréchet Inception Distance

Die FID ist eine Metrik zur Bewertung der Qualität GANs und wurde 2017 von Heusel et al. eingeführt. Sie stellt eine Weiterentwicklung des Inception score (IS) dar, dessen Schwachpunkt darin besteht, dass er die statistische Verteilung realer Bilder nicht in die Bewertung einbezieht [48, S. A1]. Die Berechnung der FID erfolgt in zwei Schritten. Zunächst werden sowohl reale als auch generierte Bilder durch ein vortrainiertes CNN geleitet [49, S. 2]. Aus einer intermediären Schicht dieses Netzwerks werden abstrakte Merkmalsvektoren gewonnen, die als visuelle Repräsentationen der Bilder dienen [50, S. 130]. Im zweiten Schritt werden diese Merkmale statistisch beschrieben. Für die realen Bilder wird der Mittelwert μ_{data} also der durchschnittliche Merkmalsvektor sowie die Kovarianzmatrix Σ_{data} berechnet, welche beschreibt, wie stark die Merkmale untereinander variieren. Dasselbe geschieht für die generierten Bilder, die durch μ_g und Σ_g beschrieben werden [50, S. 130] [48, S. A1]. Anschließend wird die Fréchet Distanz zwischen den beiden so beschriebenen Verteilungen berechnet [51, S. 2]:

$$FID = \|\mu_{data} - \mu_g\|^2 + \text{Tr}\left(\Sigma_{data} + \Sigma_g - 2\left(\Sigma_{data}\Sigma_g\right)^{1/2}\right), \quad (6.1)$$

Die Formel Gl. (6.1) besteht aus zwei Teilen. Der erste Term $\|\mu_{data} - \mu_g\|^2$ misst, wie weit die durchschnittlichen Merkmale realer und generierter Bilder voneinander abweichen. Der zweite Term vergleicht die Kovarianzmatrizen beider Verteilungen und erfasst damit, ob die generierten Bilder eine ähnliche Vielfalt und Struktur aufweisen wie die realen cite [51, S. 2]. In Abb. 6.1 wird der Prozess dargestellt. Der Generator G erzeugt Bilder, die

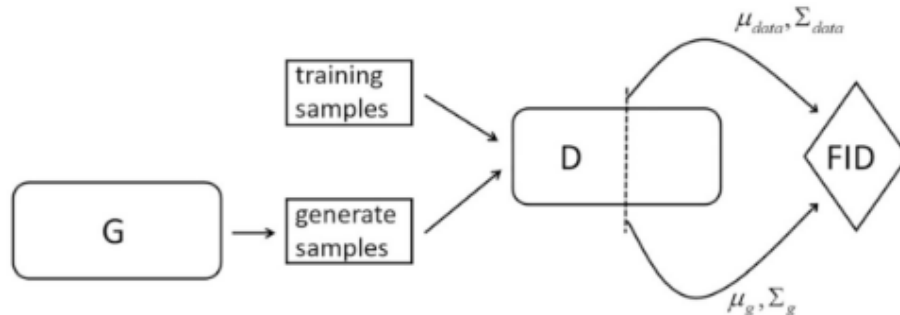


Abb. 6.1: Darstellung der FID-Berechnung [50, S. 130]

zusammen mit den Trainingsbildern durch das CNN D geleitet werden, um die statistischen Kenngrößen μ_{data} , Σ_{data} , μ_g und Σ_g zu extrahieren und daraus den FID-Wert zu

berechnen. Ein niedriger FID-Wert zeigt an, dass beide Verteilungen eng beieinanderliegen, was auf hohe Bildqualität und ausreichende Diversität hindeutet [50, S. 130].

Structural Similarity Index Measure

Der SSIM ist eine Metrik zur Bewertung von Bildqualität, die sich an der menschlichen Wahrnehmung orientiert [52, S. 1131] [47, S. 6]. Im Gegensatz zur Mean squared Error (MSE), der lediglich pixelweise Helligkeitsunterschiede misst, analysiert der SSIM, wie ähnlich zwei Bilder in ihrer Struktur sind also in den Mustern und Abhängigkeiten benachbarter Pixel, die für das menschliche Auge relevant sind [53, S. 1]. Dazu zerlegt SSIM den Bildvergleich in drei Teilkomponenten: Intensität, Kontrast und Struktur. Diese werden getrennt berechnet und anschließend multipliziert [53, S. 5 f.]. Die resultierende Formel lautet [54, S. 2] [53, S. 5] [47, S. 6] [52, S. 1134]:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (6.2)$$

In Gl. (6.2) stehen μ_x und μ_y für die mittleren Helligkeitswerte, σ_x^2 und σ_y^2 für die lokalen Varianzen und σ_{xy} für die Kovarianz also das Maß, wie ähnlich sich die Helligkeitsverläufe der beiden Bilder in einem lokalen Bereich verhalten. Die Konstanten C_1 und C_2 verhindern numerische Instabilitäten bei sehr kleinen Nennerwerten und sind als $C_1 = (K_1L)^2$ und $C_2 = (K_2L)^2$ definiert, wobei L den Wertebereich der Pixel angibt und typischerweise $K_1 = 0,01$ sowie $3K_2 = 0,03$ gewählt werden [54, S. 1 f.] [53, S. 5] [47, S. 6] [52, S. 1134]. Der berechnete SSIM-Wert eines Bildes ergibt sich als Mittelwert aller lokal berechneten Fensterwerte und liegt stets zwischen 0 und 1, wobei 1 für eine vollständige Übereinstimmung bedeutet [52, S. 1131]. Jedoch ist der SSIM eine FR-Metrik und benötigt somit ein fehlerfreies Referenzbild [53, S. 1]

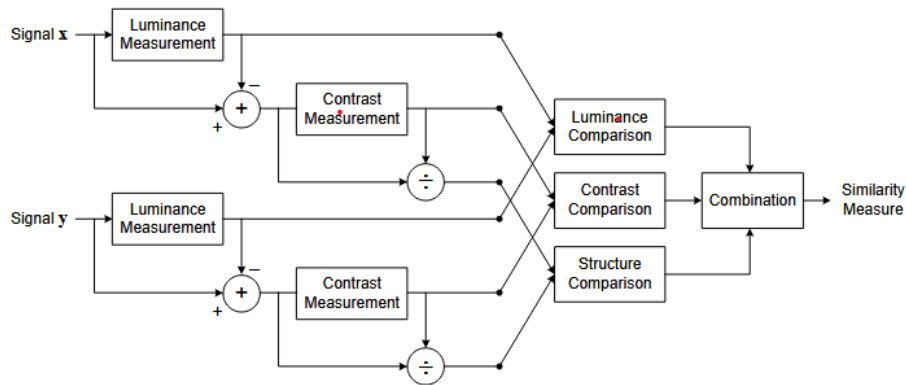


Abb. 6.2: Darstellung der SSIM-Berechnung [53, S. 5]

6.4.3 Peak Signal-to-Noise Ratio

Eine weitere Metrik ist die PSNR. PSNR bewertet die Bildqualität in dem sie das generierte Bild mit dem original Bild vergleicht [keylist].

Ein hoher PSNR Wert deutet auf eine hohe Bildqualität und desto kleiner ist die Differenz der beiden verglichen Bilder [vgl. 50, S. 137]. Allerdings tut diese Metrik nicht die menschliche Wahrnehmung abbilden, da sie lediglich sich nur auf die Pixeldifferenzen bezieht. So kann bereits eine kleine räumliche Verschiebung der Pixel zu einer Verschlechterung des PSNR-Wertes führen, obwohl die subjektiv wahrgenommene Bildqualität keinen Unterschied erkennt. Daher erzielen Modelle mit pixelbasiertem Trainingsziel höhere PSNR-Werte als generative Modelle [vgl. 55, S. 3]. Die PSNR kein ausreichendes Qualitätsmaß und sollte im Kontext generativer Modelle durch wahrnehmungsbasierte Metriken ergänzt werden.

Die PSNR ist daher zwar ein etabliertes, aber kein ausreichendes Qualitätsmaß und sollte im Kontext generativer Modelle stets durch wahrnehmungsbasierte Metriken (z. B. LPIPS, FID) ergänzt werden.

6.4.4 Learned Perceptual Image Patch Similarity

6.4.5 Dice

7 Implementierung und experimenteller Aufbau

Das folgende Kapitel beschreibt die praktische Umsetzung der in dieser Arbeit untersuchten Modelle. Zunächst werden die eingesetzten Werkzeuge und die zugrunde liegende Hardware-Konfiguration vorgestellt. Anschließend wird auf die Architektur der einzelnen Modelle sowie die Wahl der jeweiligen Hyperparameter eingegangen und deren Einfluss auf das Trainingsverhalten erläutert.

7.1 Entwicklungsumgebung und Hardware

Das Training der Modelle wurde auf einem lokalen System durchgeführt. Dieses verfügt über eine NVIDIA GeForce GTX 1080 Ti mit 11 GB VRAM, einen AMD Athlon X4 860K Prozessor (4 Kerne, 3,7 GHz), 16 GB Arbeitsspeicher sowie ein Linux-Ubuntu-Desktop-Betriebssystem. Der Einsatz einer dedizierten GPU ermöglicht eine Reduktion der Trainingszeiten gegenüber einer reinen CPU-basierten Berechnung.

Die Implementierung sämtlicher Modelle erfolgte in der Programmiersprache Python. Python besitzt eine umfangreiche Anzahl an Bibliotheken, wie Keras und TensorFlow, für maschinelles Lernen und hat sich als De-facto-Standard in diesem Bereich etabliert [56]. Als Entwicklungsumgebung wurde Visual Studio Code in Kombination mit Jupyter-Notebooks eingesetzt. Die Verwendung von Jupyter-Notebooks ermöglicht eine schrittweise Ausführung einzelner Code-Zellen, was bei der Entwicklung und dem Debugging von Vorteil ist. Für jedes Modell wurde ein separates Notebook angelegt, um eine klare Trennung der Experimente zu gewährleisten.

Für den Modellaufbau wurde das Deep-Learning-Framework PyTorch eingesetzt. Die Diffusionslogik einschließlich des Noise Schedulers, des Sampling-Verfahrens und der Trainingsschleife wurde auf Basis von PyTorch implementiert. Für das Laden und Speichern der medizinischen Bilddaten im NIfTI-Format wurde die Bibliothek nibabel verwendet. Die Visualisierung der Ergebnisse erfolgte mit Matplotlib. Die Implementierung basiert auf Python 3.12.3 und PyTorch 2.7.1 mit CUDA 11.8. Die Wahl der CUDA-Version ergibt sich aus der Tatsache, dass die NVIDIA GeForce GTX 1080 Ti auf der Pascal-Architektur (Compute Capability 6.1) basiert, welche von neueren CUDA-Versionen nicht mehr vollständig unterstützt wird.

7.2 Datenvorverarbeitung

Vor dem Training wurden die MRT-Volumina des BraTS-2023-Datensatzes vorverarbeitet. Die Vorverarbeitung orientiert sich an der Datenverarbeitung von Durrer, Cattin und Wolleb [57, S. 6]. Es umfasst fünf Schritte, die im Folgenden beschrieben werden.

Intensitätsbegrenzung. Die Intensitätswerte werden auf das Intervall zwischen dem 0,5%- und dem 99,5%-Perzentil beschnitten. Dadurch werden extreme Ausreißer entfernt, die etwa durch Aufnahmeartefakte entstehen können, ohne relevante Bildinformationen zu verlieren.

Hintergrundentfernung. Anschließend wird der Hintergrund entfernt, der hauptsächlich aus Luft und leeren Voxeln besteht. Dazu wird die kleinste Bounding Box berechnet, die alle Voxel mit einem Intensitätswert größer null enthält. Das Volumen wird so auf den anatomisch relevanten Bereich zugeschnitten.

Kubisches Padding. Da die Volumina nach dem Cropping unterschiedliche Dimensionen aufweisen, werden sie durch symmetrisches Zero-Padding auf eine kubische Form gebracht. Im Gegensatz zu einer direkten Skalierung bleibt durch das Padding das Seitenverhältnis erhalten und es werden keine geometrischen Verzerrungen eingeführt.

Resizing. Das kubische Volumen wird mittels trilinearer Interpolation auf die Zielauflösung von 32^3 oder 48^3 Voxeln skaliert. Da das Volumen bereits kubisch ist, erfolgt die Skalierung in allen drei Raumrichtungen gleichmäßig.

Normalisierung. Abschließend werden die Intensitätswerte linear auf den Wertebereich $[-1, 1]$ normalisiert. Dies ist die gängige Konvention für Diffusionsmodelle, da der Rauschprozess auf einem standardisierten Wertebereich arbeitet.

TODO: Ab hier nochm l durch

7.3 Modellarchitektur

Als Architektur dient ein 3D-UNet, das dem in der DDPM-Literatur etablierten Aufbau folgt Ho, Jain und Abbeel [22]. Die Architektur ist symmetrisch als Encoder-Decoder aufgebaut und besteht aus vier Auflösungsstufen mit Kanalbreiten von 96, 192, 384 und 384. Im Encoder wird die räumliche Auflösung schrittweise halbiert, während der Decoder sie über trilineare Interpolation wieder auf die Eingabegröße bringt. Zwischen den entsprechenden Stufen von Encoder und Decoder werden Skip-Connections eingesetzt, um räumliche Informationen direkt weiterzugeben.

Residualblöcke und Zeiteinbettung Auf jeder Auflösungsstufe befinden sich zwei Residualblöcke, die jeweils aus zwei 3D-Faltungsschichten mit GroupNorm und SiLU-Aktivierung bestehen. Der aktuelle Diffusions-Zeitschritt wird über sinusoidale Positionseinbettungen kodiert und über eine lineare Projektion in jeden Residualblock eingespeist, sodass das Netzwerk sein Verhalten an den jeweiligen Rauschgrad anpassen kann.

Self-Attention. Auf den beiden niedrigsten Auflösungsstufen werden zusätzlich Self-Attention-Schichten eingesetzt, die es dem Netzwerk ermöglichen, globale Abhängigkeiten zwischen räumlich entfernten Regionen zu modellieren. Auf höheren Auflösungen wird darauf verzichtet, da der Speicherbedarf mit der Voxelanzahl ansteigt.

7.4 Baseline-Konfiguration

Die Baseline-Konfiguration benutzt mehrere Techniken die sich in der aktuellen Literatur als Verbesserungen gegenüber dem ursprünglichen DDPM-Ansatz von Ho et al. [ho2020ddpm] etabliert haben. Sie dient als Referenzmodell, gegen das alle weiteren Experimente verglichen werden.

Der Diffusionsprozess verwendet 250 Zeitschritte mit einem Cosine Schedule [nichol2021improved]. Im Vergleich zum linearen Schedule verteilt der Cosine Schedule das Signal-Rausch-Verhältnis gleichmäßiger über die Zeitschritte was insbesondere bei niedrigen Auflösungen zu einer höheren Bildqualität führt [nichol2021improved].

Das Modell wird mit v-Prediction trainiert [salimans2022progressive]. Dabei sagt das Netzwerk weder das Rauschen ϵ noch das ursprüngliche Bild x_0 direkt vorher, sondern eine Kombination aus beiden, die als Velocity v bezeichnet wird. Dieses Vorhersageziel hat sich als numerisch stabiler erwiesen, insbesondere bei niedrigen Signal-Rausch-Verhältnissen [salimans2022progressive].

Für die Gewichtung des Trainingsverlusts wird Min-SNR Weighting mit $\gamma = 5$ eingesetzt [hang2023minsnr]. Diese Strategie begrenzt den Einfluss von Zeitschritten mit hohem Signal-Rausch-Verhältnis auf den Gesamtverlust und sorgt so für ein ausgewogeneres Training über alle Zeitschritte hinweg.

7.5 Trainingsverfahren und Hyperparameter

Alle Modelle werden über 300 Epochen mit dem Adam-Optimizer und einer Lernrate von 2×10^{-4} trainiert. Diese Lernrate entspricht dem in der DDPM-Literatur üblichen Wert [ho2020ddpm]. In den ersten 500 Iterationen wird die Lernrate linear von null auf den Zielwert erhöht (Warmup), um Instabilitäten zu Beginn des Trainings zu vermeiden. Die Batch Size beträgt 10, was dem verfügbaren GPU-Speicher von 11 GB geschuldet ist.

Zur Stabilisierung der Modellgewichte wird Exponential Moving Average (EMA) mit einem Decay-Faktor von 0,9999 eingesetzt [nichol2021improved]. Dabei wird parallel zu

den trainierten Gewichten ein gleitender Durchschnitt geführt, der für die finale Evaluation und das Sampling verwendet wird. Zusätzlich wird Gradient Clipping mit einer maximalen Norm von 1,0 angewendet, um Gradientenexplosionen zu verhindern.

Der Datensatz wird in 90% Trainings- und 10% Validierungsdaten aufgeteilt. Die Aufteilung erfolgt mit einem festen Random Seed, um die Reproduzierbarkeit sicherzustellen. Während des Trainings wird der Validierungsverlust nach jeder Epoche berechnet.

7.6 Ablationsstudien

Um den Einfluss einzelner Komponenten auf die Generierungsqualität zu untersuchen werden ausgehend von der Baseline-Konfiguration gezielt einzelne Variablen verändert. Pro Experiment wird jeweils nur eine Komponente angepasst, um deren isolierten Einfluss bewerten zu können. Die Ablationsstudien werden auf einer Auflösung von 32^3 Voxeln durchgeführt.

Ohne Self-Attention. Die Self-Attention-Schichten auf den niedrigen Auflösungsstufen werden entfernt. Dadurch lässt sich untersuchen, welchen Beitrag die Modellierung globaler Abhängigkeiten zur Bildqualität leistet und ob die lokalen Informationen der Faltungsschichten allein ausreichen.

Linearer Schedule. Der Cosine Schedule wird durch einen linearen Schedule ersetzt, bei dem die Varianz gleichmäßig von $\beta_1 = 0,0001$ bis $\beta_T = 0,02$ ansteigt. Dies entspricht dem ursprünglichen Vorschlag von Ho et al. [ho2020ddpm].

Noise-Prediction (ϵ -Prediction). Anstelle der v-Prediction wird das Modell darauf trainiert, das hinzugefügte Rauschen ϵ direkt vorherzusagen. Dies ist die ursprüngliche DDPM-Formulierung [ho2020ddpm] und dient als Vergleich zur moderneren v-Prediction.

Datenaugmentierung. Die Trainingsdaten werden durch zufällige räumliche Transformationen augmentiert, um die effektive Datensatzgröße zu erhöhen und die Generalisierung zu verbessern.

Erhöhte Auflösung (48^3). Zusätzlich wird die Baseline-Konfiguration auf einer Auflösung von 48^3 Voxeln trainiert, um den Einfluss der räumlichen Auflösung auf die Bildqualität zu untersuchen. Aufgrund der deutlich längeren Trainingszeiten bei höherer Auflösung werden die übrigen Ablationen nur auf 32^3 durchgeführt.

7.7 Aufgezeichnete Werte

Während des Trainings werden für jede Epoche verschiedene Metriken protokolliert, um den Trainingsverlauf nachvollziehen zu können. Dazu gehören der Trainings- und Validierungsverlust, die aktuelle Lernrate, die Epochendauer sowie der GPU-Speicherverbrauch. Zusätzlich werden die Gradientennorm und die Anzahl der durch Gradient Clipping begrenzten Updates erfasst, um die Stabilität des Trainings beurteilen zu können.

In regelmäßigen Abständen von 25 Epochen werden Samples generiert, um die visuelle Qualität der erzeugten Volumina im Trainingsverlauf zu überprüfen. Checkpoints werden an festgelegten Epochen gespeichert sowie für das Modell mit dem besten Validierungsverlust. Sämtliche Metriken werden nach jeder Epoche in einer JSON-Datei gespeichert, sodass der Trainingsverlauf auch nachträglich vollständig analysiert werden kann.

7.8 Projektmethodik

TODO: Kanban oder sowas

8 Ergebnis

9 Schlussfolgerung

9.1 Fazit

9.2 Ausblick

Literatur

- [1] Sridaran Rajagopal u. a., Hrsg. *Artificial Intelligence Based Smart and Secured Applications: 4th International Conference, ASCIS 2025, Gujarat, India, September 11–13, 2025, Revised Selected Papers, Part III*. en. Bd. 2821. Communications in Computer and Information Science. Cham: Springer Nature Switzerland, 2026. ISBN: 978-3-032-17839-8 978-3-032-17840-4. DOI: 10.1007/978-3-032-17840-4. URL: <https://link.springer.com/10.1007/978-3-032-17840-4> (besucht am 18.02.2026).
- [2] Ahmed Elhadad, Mona Jamjoom und Hussein Abulkasim. „Reduction of NIFTI files storage and compression to facilitate telemedicine services based on quantization hiding of downsampling approach“. en. In: *Scientific Reports* 14.1 (März 2024), S. 5168. ISSN: 2045-2322. DOI: 10.1038/s41598-024-54820-4. URL: <https://www.nature.com/articles/s41598-024-54820-4> (besucht am 18.02.2026).
- [3] Department of Computer Applications, Kalasalingam Academy of Research and Education (Deemed to be University), Krishnankoil, Tamil Nadu, India. u. a. „An Medical Image File Formats and Digital Image Conversion“. en. In: *International Journal of Engineering and Advanced Technology* 9.1s4 (Dez. 2019), S. 74–78. ISSN: 22498958. DOI: 10.35940/ijeat.A1093.1291S419. URL: <https://www.ijeat.org/portfolio-item/A10931291S419/> (besucht am 18.02.2026).
- [4] Harald Nahrstedt. *Algorithmen für Ingenieure*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2018. ISBN: 978-3-658-19298-3 978-3-658-19299-0. DOI: 10.1007/978-3-658-19299-0. URL: <http://link.springer.com/10.1007/978-3-658-19299-0> (besucht am 10.10.2025).
- [5] Zhen LLeo" Liu. *Artificial Intelligence for Engineers: Basics and Implementations*. en. Cham: Springer Nature Switzerland, 2025. ISBN: 978-3-031-75952-9 978-3-031-75953-6. DOI: 10.1007/978-3-031-75953-6. URL: <https://link.springer.com/10.1007/978-3-031-75953-6> (besucht am 06.10.2025).
- [6] Andreas Kohne u. a. *Chatbots: Aufbau und Anwendungsmöglichkeiten von autonomen Sprachassistenten*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2020. ISBN: 978-3-658-28848-8 978-3-658-28849-5. DOI: 10.1007/978-3-658-28849-5. URL: <http://link.springer.com/10.1007/978-3-658-28849-5> (besucht am 06.10.2025).
- [7] Vasant Honavar. „Artificial Intelligence: An Overview“. en. In: (). (Besucht am 07.10.2025).
- [8] Stuart J. Russell und Peter Norvig. *Artificial intelligence: a modern approach*. en. Third edition, Global edition. Prentice Hall series in artificial intelligence. Boston

- Columbus Indianapolis: Pearson, 2016. ISBN: 978-0-13-604259-4 978-1-292-15397-1. (Besucht am 07. 10. 2025).
- [9] Paolo Bory, Simone Natale und Christian Katzenbach. „Strong and weak AI narratives: an analytical framework“. en. In: *AI & SOCIETY* 40.4 (Apr. 2025), S. 2107–2117. ISSN: 0951-5666, 1435-5655. DOI: 10.1007/s00146-024-02087-8. URL: <https://link.springer.com/10.1007/s00146-024-02087-8> (besucht am 14. 10. 2025).
- [10] Carsten Lanquillon und Sigurd Schacht. *Knowledge Science - Grundlagen: Methoden der Künstlichen Intelligenz für die Wissensextraktion aus Texten*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2023. ISBN: 978-3-658-41688-1 978-3-658-41689-8. DOI: 10.1007/978-3-658-41689-8. URL: <https://link.springer.com/10.1007/978-3-658-41689-8> (besucht am 06. 10. 2025).
- [11] Robert Ciesla. *The Book of Chatbots: From ELIZA to ChatGPT*. en. Cham: Springer Nature Switzerland, 2024. ISBN: 978-3-031-51003-8 978-3-031-51004-5. DOI: 10.1007/978-3-031-51004-5. URL: <https://link.springer.com/10.1007/978-3-031-51004-5> (besucht am 14. 10. 2025).
- [12] Nils Urbach und Daniel Feulner, Hrsg. *Managing Artificial Intelligence: How Organizations Succeed with AI*. en. Future of Business and Finance. Cham: Springer Nature Switzerland, 2026. ISBN: 978-3-032-13307-6 978-3-032-13308-3. DOI: 10.1007/978-3-032-13308-3. URL: <https://link.springer.com/10.1007/978-3-032-13308-3> (besucht am 24. 03. 2026).
- [13] Tom M. Mitchell. *Machine learning*. en. Nachdr. McGraw-Hill series in Computer Science. New York: McGraw-Hill, 2013. ISBN: 978-0-07-042807-2 978-0-07-115467-3.
- [14] F. Rosenblatt. *The Perceptron, a Perceiving and Recognizing Automaton: (Project Para)*. Report / Cornell Aeronautical Laboratory. Cornell Aeronautical Laboratory, 1957. URL: https://books.google.de/books?id=P_XGPgAACAAJ.
- [15] Hang Li. *Machine Learning Methods*. en. Singapore: Springer Nature Singapore, 2024. ISBN: 978-981-99-3916-9 978-981-99-3917-6. DOI: 10.1007/978-981-99-3917-6. URL: <https://link.springer.com/10.1007/978-981-99-3917-6> (besucht am 07. 04. 2026).
- [16] Andrew Y. Ng und Michael I. Jordan. „On Discriminative vs. Generative Classifiers: A Comparison of Logistic Regression and Naive Bayes“. In: *Advances in Neural Information Processing Systems 14 (NIPS 2001)*. Hrsg. von Thomas G. Dietterich, Suzanna Becker und Zoubin Ghahramani. Cambridge, MA, USA: MIT Press, 2002, S. 841–848.
- [17] Christopher M. Bishop. *Pattern recognition and machine learning*. en. Information science and statistics. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
- [18] Stefan Feuerriegel u. a. „Generative AI“. en. In: *Business & Information Systems Engineering* 66.1 (Feb. 2024), S. 111–126. ISSN: 2363-7005, 1867-0202. DOI: 10.1007/

- s12599-023-00834-7. URL: <https://link.springer.com/10.1007/s12599-023-00834-7> (besucht am 14.02.2026).
- [19] Chieh-Hsin Lai u. a. *The Principles of Diffusion Models*. en. arXiv:2510.21890 [cs]. Okt. 2025. DOI: 10.48550/arXiv.2510.21890. URL: <http://arxiv.org/abs/2510.21890> (besucht am 14.02.2026).
- [20] Lars Ruthotto und Eldad Haber. „An introduction to deep generative modeling“. en. In: *GAMM-Mitteilungen* 44.2 (Juni 2021), e202100008. ISSN: 0936-7195, 1522-2608. DOI: 10.1002/gamm.202100008. URL: <https://onlinelibrary.wiley.com/doi/10.1002/gamm.202100008> (besucht am 15.02.2026).
- [21] Takeshi Okadome. *Essentials of Generative AI*. en. Singapore: Springer Nature Singapore, 2025. ISBN: 978-981-96-0028-1 978-981-96-0029-8. DOI: 10.1007/978-981-96-0029-8. URL: <https://link.springer.com/10.1007/978-981-96-0029-8> (besucht am 15.02.2026).
- [22] Jonathan Ho, Ajay Jain und Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. en. arXiv:2006.11239 [cs]. Dez. 2020. DOI: 10.48550/arXiv.2006.11239. URL: <http://arxiv.org/abs/2006.11239> (besucht am 15.02.2026).
- [23] Alex Nichol und Prafulla Dhariwal. *Improved Denoising Diffusion Probabilistic Models*. en. arXiv:2102.09672 [cs]. Feb. 2021. DOI: 10.48550/arXiv.2102.09672. URL: <http://arxiv.org/abs/2102.09672> (besucht am 17.03.2026).
- [24] Jiaming Song, Chenlin Meng und Stefano Ermon. *Denoising Diffusion Implicit Models*. en. arXiv:2010.02502 [cs]. Okt. 2022. DOI: 10.48550/arXiv.2010.02502. URL: <http://arxiv.org/abs/2010.02502> (besucht am 19.03.2026).
- [25] Cheng Lu u. a. „DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models“. en. In: *Machine Intelligence Research* 22.4 (Aug. 2025). arXiv:2211.01095 [cs], S. 730–751. ISSN: 2731-538X, 2731-5398. DOI: 10.1007/s11633-025-1562-4. URL: <http://arxiv.org/abs/2211.01095> (besucht am 04.05.2026).
- [26] Wenliang Zhao u. a. „UniPC: A Unified Predictor-Corrector Framework for Fast Sampling of Diffusion Models“. en. In: ().
- [27] Keiron O’Shea und Ryan Nash. *An Introduction to Convolutional Neural Networks*. en. arXiv:1511.08458 [cs]. Dez. 2015. DOI: 10.48550/arXiv.1511.08458. URL: <http://arxiv.org/abs/1511.08458> (besucht am 04.03.2026).
- [28] Nhat-Duc Hoang und Van-Duc Tran. „Deep Learning-Based Predictive Modeling of Urban Heat Stress Using Transformer Network, Deep Neural Network, and Convolutional Neural Network“. en. In: *Remote Sensing in Earth Systems Sciences* 8.4 (Dez. 2025), S. 1161–1184. ISSN: 2520-8195, 2520-8209. DOI: 10.1007/s41976-025-00242-3. URL: <https://link.springer.com/10.1007/s41976-025-00242-3> (besucht am 04.03.2026).
- [29] Wei Shen u. a. *Hands-On Computer Vision*. en. Singapore: Springer Nature Singapore, 2026. ISBN: 9789819548347 9789819548354. DOI: 10.1007/978-981-95-

- 4835-4. URL: <https://link.springer.com/10.1007/978-981-95-4835-4> (besucht am 06.03.2026).
- [30] Alex Krizhevsky, Ilya Sutskever und Geoffrey E. Hinton. „ImageNet classification with deep convolutional neural networks“. en. In: *Communications of the ACM* 60.6 (Mai 2017), S. 84–90. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3065386. URL: <https://dl.acm.org/doi/10.1145/3065386> (besucht am 04.03.2026).
- [31] Marvin John Ignacio u. a. „Revisiting U-Net: a foundational backbone for modern generative AI“. en. In: *Artificial Intelligence Review* 59.2 (Nov. 2025), S. 45. ISSN: 1573-7462. DOI: 10.1007/s10462-025-11450-0. URL: <https://link.springer.com/10.1007/s10462-025-11450-0> (besucht am 24.02.2026).
- [32] Olaf Ronneberger, Philipp Fischer und Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. en. arXiv:1505.04597 [cs]. Mai 2015. DOI: 10.48550/arXiv.1505.04597. URL: <http://arxiv.org/abs/1505.04597> (besucht am 22.02.2026).
- [33] Nicholas Metropolis und S. Ulam. „The Monte Carlo Method“. In: *Journal of the American Statistical Association* 44.247 (1949). PMID: 18139350, S. 335–341. DOI: 10.1080/01621459.1949.10483310. eprint: <https://www.tandfonline.com/doi/pdf/10.1080/01621459.1949.10483310>. URL: <https://www.tandfonline.com/doi/abs/10.1080/01621459.1949.10483310>.
- [34] James Jordon u. a. *Synthetic Data - what, why and how?* 2022. arXiv: 2205.03257 [cs.LG]. URL: <https://arxiv.org/abs/2205.03257>.
- [35] Mauro Giuffrè und Dennis L. Shung. „Harnessing the power of synthetic data in healthcare: innovation, application, and privacy“. en. In: *npj Digital Medicine* 6.1 (Okt. 2023), S. 186. ISSN: 2398-6352. DOI: 10.1038/s41746-023-00927-3. URL: <https://www.nature.com/articles/s41746-023-00927-3> (besucht am 09.04.2026).
- [36] Bronwyn Loong u. a. „Disclosure control using partially synthetic data for large scale health surveys, with applications to CanCORS“. en. In: *Statistics in Medicine* 32.24 (Okt. 2013), S. 4139–4161. ISSN: 0277-6715, 1097-0258. DOI: 10.1002/sim.5841. URL: <https://onlinelibrary.wiley.com/doi/10.1002/sim.5841> (besucht am 09.04.2026).
- [37] T E Raghunathan, J P Reiter und D B Rubin. „Multiple Imputation for Statistical Disclosure Limitation“. en. In: ().
- [38] *Art. 9 DSGVO – Verarbeitung besonderer Kategorien personenbezogener Daten*. 2018. URL: <https://dsgvo-gesetz.de/art-9-dsgvo/> (besucht am 18.04.2026).
- [39] *Art. 6 KI-VO – Einstufungsvorschriften für Hochrisiko-KI-Systeme*. 2024. URL: <https://artificialintelligenceact.eu/de/article/6/> (besucht am 18.04.2026).
- [40] *Art. 10 KI-VO – Daten und Daten-Governance*. 2024. URL: <https://artificialintelligenceact.eu/de/article/10/> (besucht am 18.04.2026).
- [41] Maja Nisevic, Dusko Milojevic und Daniela Spajic. „Synthetic data in medicine: Legal and ethical considerations for patient profiling“. en. In: *Computational and*

- Structural Biotechnology Journal* 28 (Jan. 2025), S. 190–198. ISSN: 2001-0370. DOI: 10.1016/j.csbj.2025.05.026. URL: <https://spj.science.org/doi/10.1016/j.csbj.2025.05.026> (besucht am 18.04.2026).
- [42] Shanshan Wang u. a. „Generative Artificial Intelligence in Medical Imaging: Foundations, Progress, and Clinical Translation“. en. In: *Research* 8 (Jan. 2025), S. 1029. ISSN: 2639-5274. DOI: 10.34133/research.1029. URL: <https://spj.science.org/doi/10.34133/research.1029> (besucht am 19.04.2026).
- [43] Firas Khader u. a. *Medical Diffusion: Denoising Diffusion Probabilistic Models for 3D Medical Image Generation*. en. arXiv:2211.03364 [eess]. Jan. 2023. DOI: 10.48550/arXiv.2211.03364. URL: <http://arxiv.org/abs/2211.03364> (besucht am 19.04.2026).
- [44] Fei Wang und Anita Preininger. „AI in Health: State of the Art, Challenges, and Future Directions“. en. In: *Yearbook of Medical Informatics* 28.01 (Aug. 2019), S. 016–026. ISSN: 0943-4747, 2364-0502. DOI: 10.1055/s-0039-1677908. URL: <http://www.thieme-connect.de/DOI/DOI?10.1055/s-0039-1677908> (besucht am 19.04.2026).
- [45] Shenghuan Sun u. a. *Aligning Synthetic Medical Images with Clinical Knowledge using Human Feedback*. en. arXiv:2306.12438 [eess]. Juni 2023. DOI: 10.48550/arXiv.2306.12438. URL: <http://arxiv.org/abs/2306.12438> (besucht am 19.04.2026).
- [46] Gregory Schuit, Denis Parra und Cecilia Besa. *Perceptual Evaluation of GANs and Diffusion Models for Generating X-rays*. en. arXiv:2508.07128 [cs]. Aug. 2025. DOI: 10.48550/arXiv.2508.07128. URL: <http://arxiv.org/abs/2508.07128> (besucht am 19.04.2026).
- [47] Pinar Civicioglu und Erkan Besdok. „Image discrimination robustness of classical image quality metrics: an analysis on MAE, MSE, ERGAS, LMSE, SSIM, SAM, UIQI, PSNR, NIQE, PIQE, SNR, VIF, FSIM, GMSD, and NCC“. en. In: *Quality & Quantity* (Okt. 2025). ISSN: 0033-5177, 1573-7845. DOI: 10.1007/s11135-025-02404-3. URL: <https://link.springer.com/10.1007/s11135-025-02404-3> (besucht am 01.03.2026).
- [48] Martin Heusel u. a. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. en. arXiv:1706.08500 [cs]. Jan. 2018. DOI: 10.48550/arXiv.1706.08500. URL: <http://arxiv.org/abs/1706.08500> (besucht am 01.03.2026).
- [49] Christian Szegedy u. a. *Rethinking the Inception Architecture for Computer Vision*. en. arXiv:1512.00567 [cs]. Dez. 2015. DOI: 10.48550/arXiv.1512.00567. URL: <http://arxiv.org/abs/1512.00567> (besucht am 01.03.2026).
- [50] Peng Long und Xiaozhou Guo. *Generative Adversarial Network: Principle and Practice*. en. Singapore: Springer Nature Singapore, 2026. ISBN: 978-981-96-9403-7 978-

- 981-96-9404-4. DOI: 10.1007/978-981-96-9404-4. URL: <https://link.springer.com/10.1007/978-981-96-9404-4> (besucht am 01.03.2026).
- [51] D. C. Dowson und B. V. Landau. „The Fréchet Distance between Multivariate Normal Distributions“. In: *Journal of Multivariate Analysis* 12.3 (1982), S. 450–455. ISSN: 0047-259X. DOI: 10.1016/0047-259X(82)90077-X.
- [52] Sho Maruyama. „Properties of the SSIM metric in medical image assessment: correspondence between measurements and the spatial frequency spectrum“. en. In: *Physical and Engineering Sciences in Medicine* 46.3 (Sep. 2023), S. 1131–1141. ISSN: 2662-4729, 2662-4737. DOI: 10.1007/s13246-023-01280-1. URL: <https://link.springer.com/10.1007/s13246-023-01280-1> (besucht am 02.03.2026).
- [53] Zhou Wang u. a. „Image quality assessment: from error visibility to structural similarity“. en. In: *IEEE Transactions on Image Processing* 13.4 (Apr. 2004), S. 600–612. ISSN: 1057-7149, 1941-0042. DOI: 10.1109/TIP.2003.819861. URL: <https://ieeexplore.ieee.org/document/1284395/> (besucht am 02.03.2026).
- [54] Jim Nilsson und Tomas Akenine-Möller. *Understanding SSIM*. en. arXiv:2006.13846 [eess]. Juni 2020. DOI: 10.48550/arXiv.2006.13846. URL: <http://arxiv.org/abs/2006.13846> (besucht am 02.03.2026).
- [55] Brian B. Moser u. a. „Diffusion Models, Image Super-Resolution And Everything: A Survey“. en. In: *IEEE Transactions on Neural Networks and Learning Systems* 36.7 (Juli 2025). arXiv:2401.00736 [cs], S. 11793–11813. ISSN: 2162-237X, 2162-2388. DOI: 10.1109/TNNLS.2024.3476671. URL: <http://arxiv.org/abs/2401.00736> (besucht am 23.04.2026).
- [56] GeeksforGeeks. *Why is Python the Best-Suited Programming Language for Machine Learning?* <https://www.geeksforgeeks.org/blogs/why-is-python-the-best-suited-programming-language-for-machine-learning/>. Besucht am: 15.03.2026. Aug. 2025.
- [57] Alicia Durrer, Philippe C. Cattin und Julia Wolleb. *Denoising Diffusion Models for Inpainting of Healthy Brain Tissue*. en. arXiv:2402.17307 [eess]. Okt. 2024. DOI: 10.48550/arXiv.2402.17307. URL: <http://arxiv.org/abs/2402.17307> (besucht am 15.03.2026).

A Anhang A

Weitere Abbildungen